

**MBA IN SOFTWARE
ENGINEERING**

DESIGN PATTERNS II

Prof. Dr. Rosana T. Vaccare Braga

MBAUSP
ESALQ

The trustworthiness, originality,
and legality of the educational content presented is responsibility of the professor.

The total or partial **reproduction** of this material without authorization is prohibited.

Law No.
9610/98

In the previous class ...

- Software reuse and its benefits
- Patterns and their origin
- Elements of a pattern, types of patterns
- Design patterns: definitions, advantages, categories, writing formats
- GoF patterns from the “creational” category and 2 patterns from the “structural” category

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adappter	Interpretador Template Method
	Object	• Factory • Abstract • Builder • Prototype • Object • Unit (Singleton)	• Adapter • Bridge • Composite • Decorator • Facade • Flyweight • Proxy	• Chain of Responsibility • Command • Iterator • Mediator* • Memento* • Observer • State • Strategy • Visitor *

In today's class

- Other patterns from the “structural” category;
- Patterns from the “behavioral” category;
- Relationship between design patterns;
- Conclusions
- Discussion: Impact of Artificial Intelligence on design patterns

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adappter	Interpretador Template Method
	Object	• Factory • Abstract • Builder • Prototype • Object • Unit (Singleton)	• Adapter • Bridge • Composite • Decorator • Facade • Flyweight • Proxy	• Chain of Responsibility • Command • Iterator • Mediator* • Memento* • Observer • State • Strategy • Visitor *

Today's class:

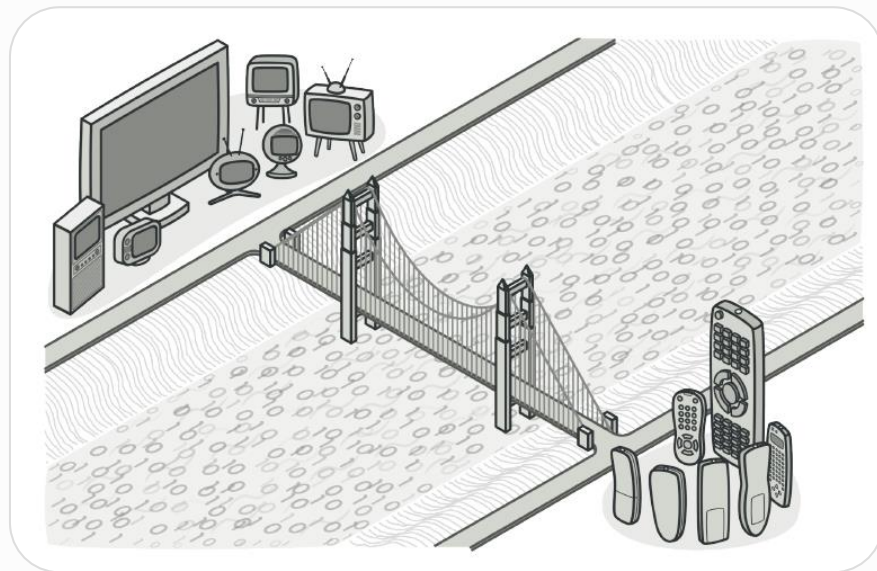
		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template Method
	Object	• Abstract Factory • Builder • Prototype • Singleton	• Adapter • Bridge • Composite • Decorator • Facade • Flyweight • Proxy	• Chain of Responsibility • Command • Iterator • Mediator • Memento • Observer • State • Strategy • Visitor

PATTERNS of the “structural” category

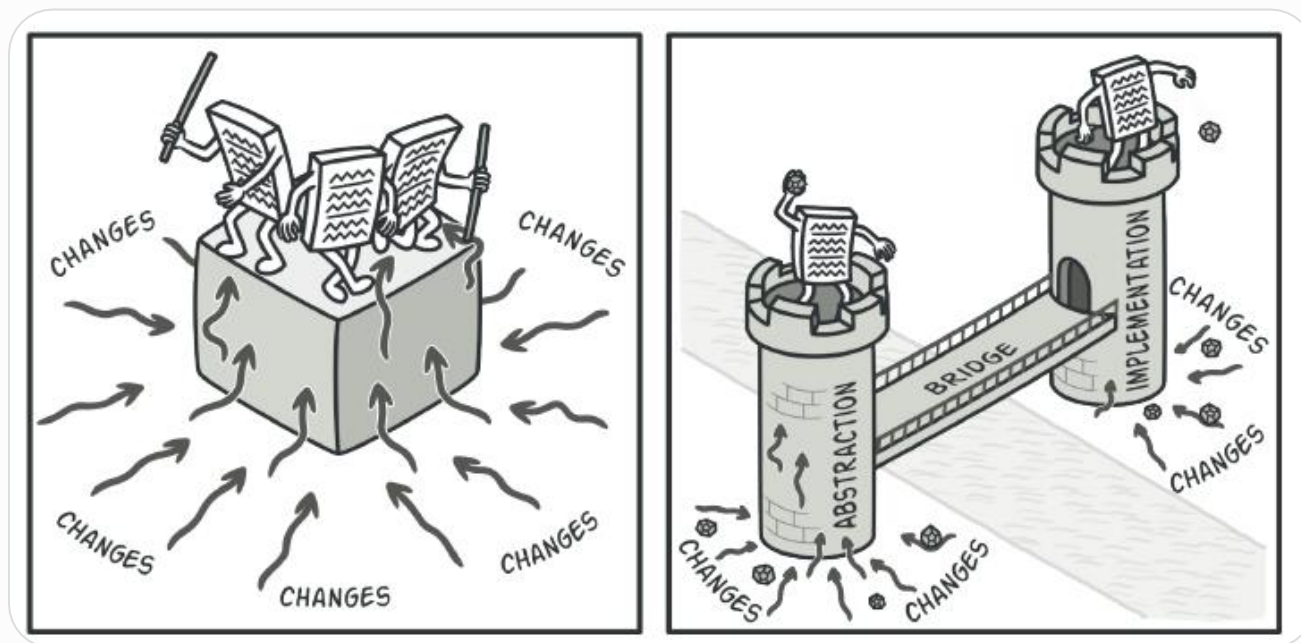
They are concerned with how compositions of classes and objects occur in larger structures

Resuming from the previous class: Bridge Design Pattern

- **Classification:** Structural pattern/Object scope
- **Intent:** Decouple an abstraction from its implementation so that the two can vary independently.



(Shvets, Alexander, 2018)



Bridge Design Pattern

- Structure:

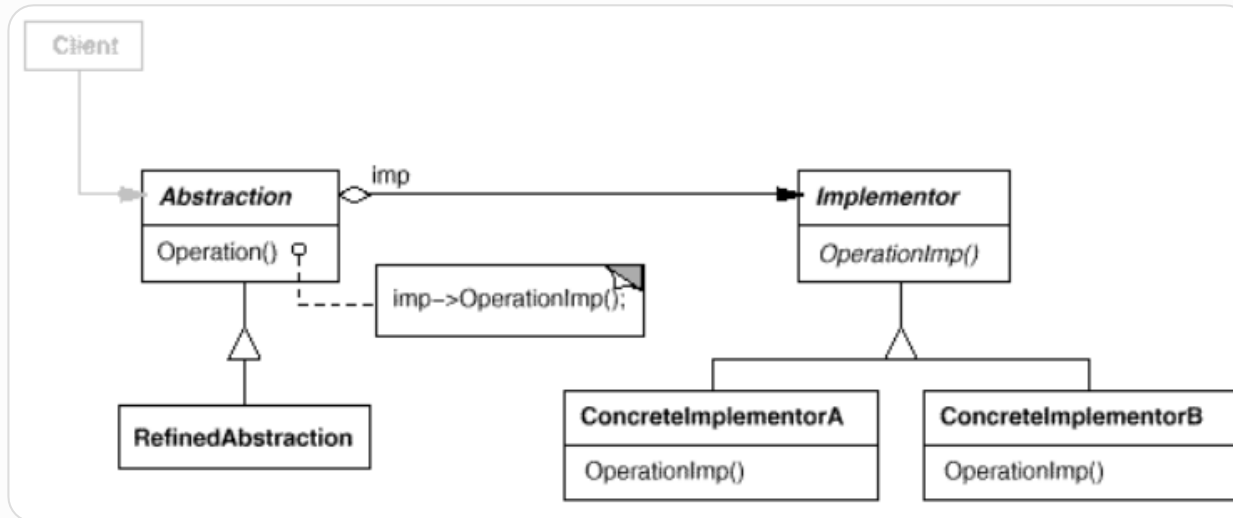
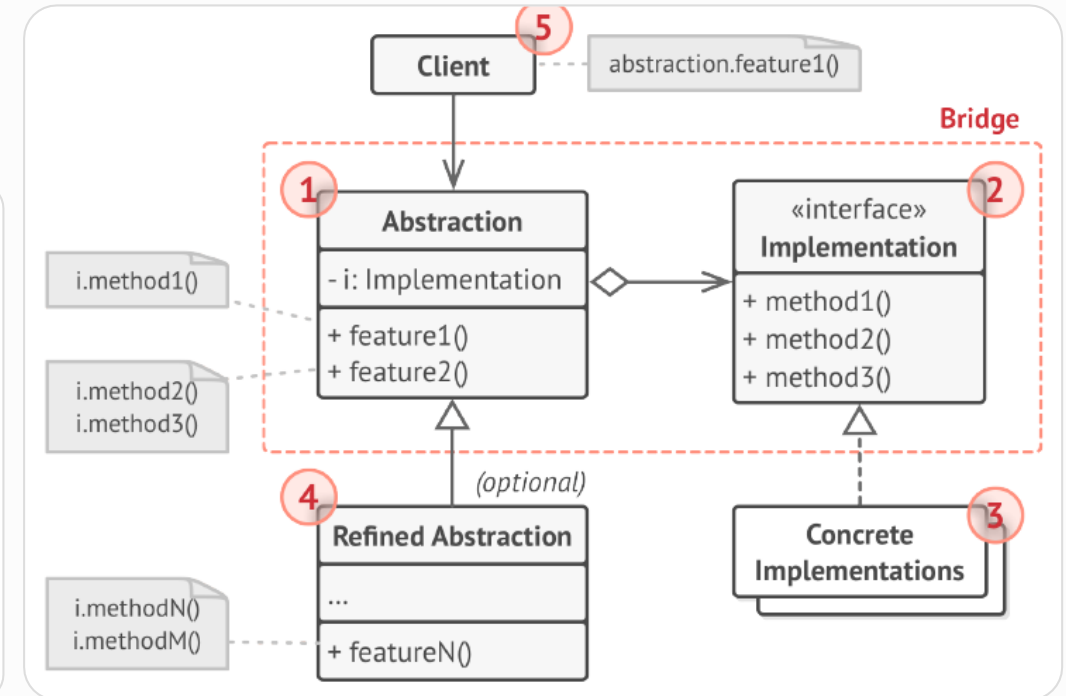


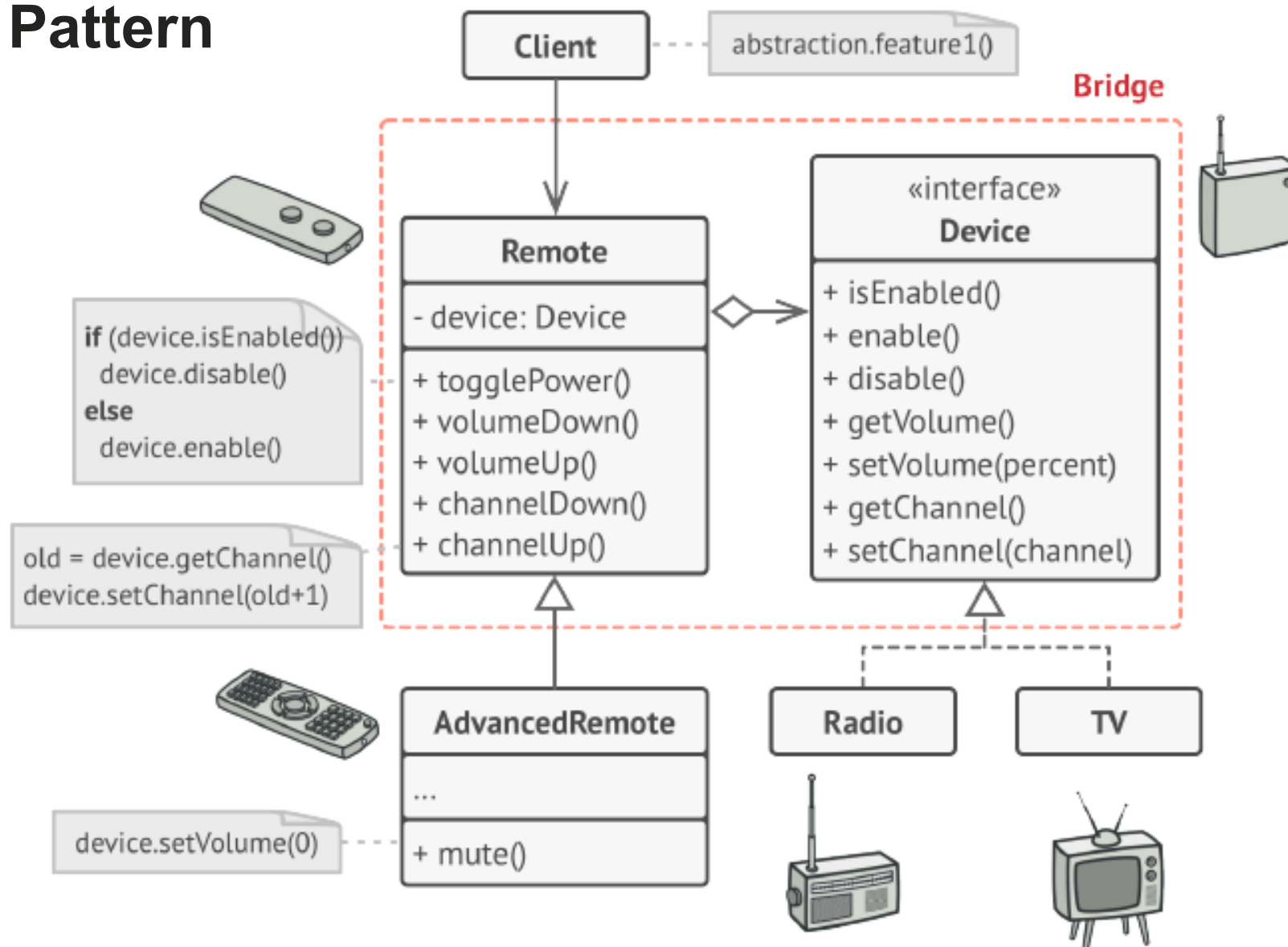
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Bridge Design Pattern

- Example:



(Shvets, Alexander, 2018)

Bridge Design Pattern

- Example:

```
23
24 // You can extend classes from the abstraction hierarchy
25 // independently from device classes.
26 class AdvancedRemoteControl extends RemoteControl is
27     method mute() is
28         device.setVolume(0)
29
```

```
1 // The "abstraction" defines the interface for the "control"
2 // part of the two class hierarchies. It maintains a reference
3 // to an object of the "implementation" hierarchy and delegates
4 // all of the real work to this object.
5 class RemoteControl is
6     protected field device: Device
7     constructor RemoteControl(device: Device) is
8         this.device = device
9     method togglePower() is
10         if (device.isEnabled()) then
11             device.disable()
12         else
13             device.enable()
14     method volumeDown() is
15         device.setVolume(device.getVolume() - 10)
16     method volumeUp() is
17         device.setVolume(device.getVolume() + 10)
18     method channelDown() is
19         device.setChannel(device.getChannel() - 1)
20     method channelUp() is
21         device.setChannel(device.getChannel() + 1)
22
```

(Shvets, Alexander, 2018)

Bridge Design Pattern

- Example:

```
37 interface Device is
38     method isEnabled()
39     method enable()
40     method disable()
41     method getVolume()
42     method setVolume(percent)
43     method getChannel()
44     method setChannel(channel)
45
46
47 // All devices follow the same interface.
48 class Tv implements Device is
49     // ...
50
51 class Radio implements Device is
52     // ...
```

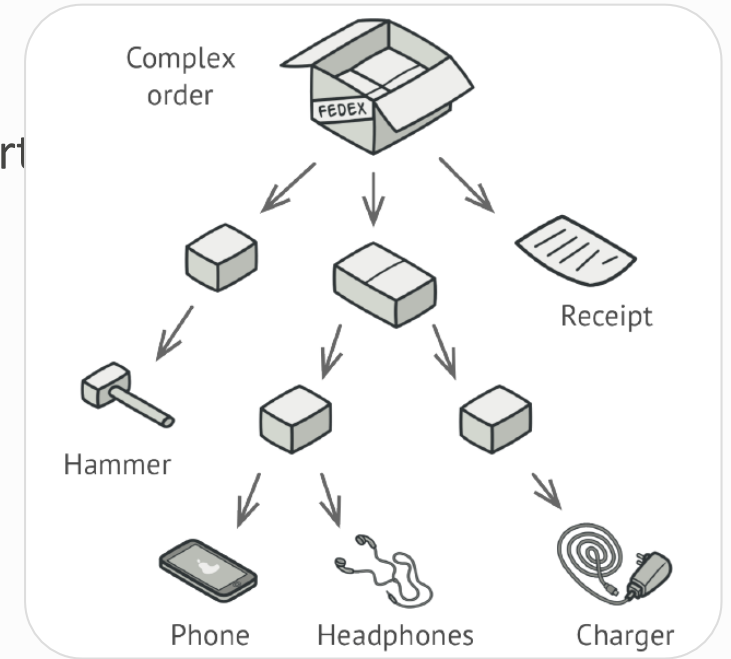
Bridge Design Pattern

- Example:

```
55 // Somewhere in client code.  
56 tv = new Tv()  
57 remote = new RemoteControl(tv)  
58 remote.togglePower()  
59  
60 radio = new Radio()  
61 remote = new AdvancedRemoteControl(radio)
```

Composite Design Pattern

- **Classification:** Creational Pattern/Object scope
- **Objective:** Compose objects into tree structures to represent whole-part hierarchies. The Composite lets clients treat individual objects and compositions of objects uniformly.
- **Motivation:** In graphical applications, you can build complex diagrams from simple components. You can group components to create other larger components, at various levels.
 - The problem is that code using these classes must treat primitive objects and objects that contain other objects differently, which makes the code more complex.

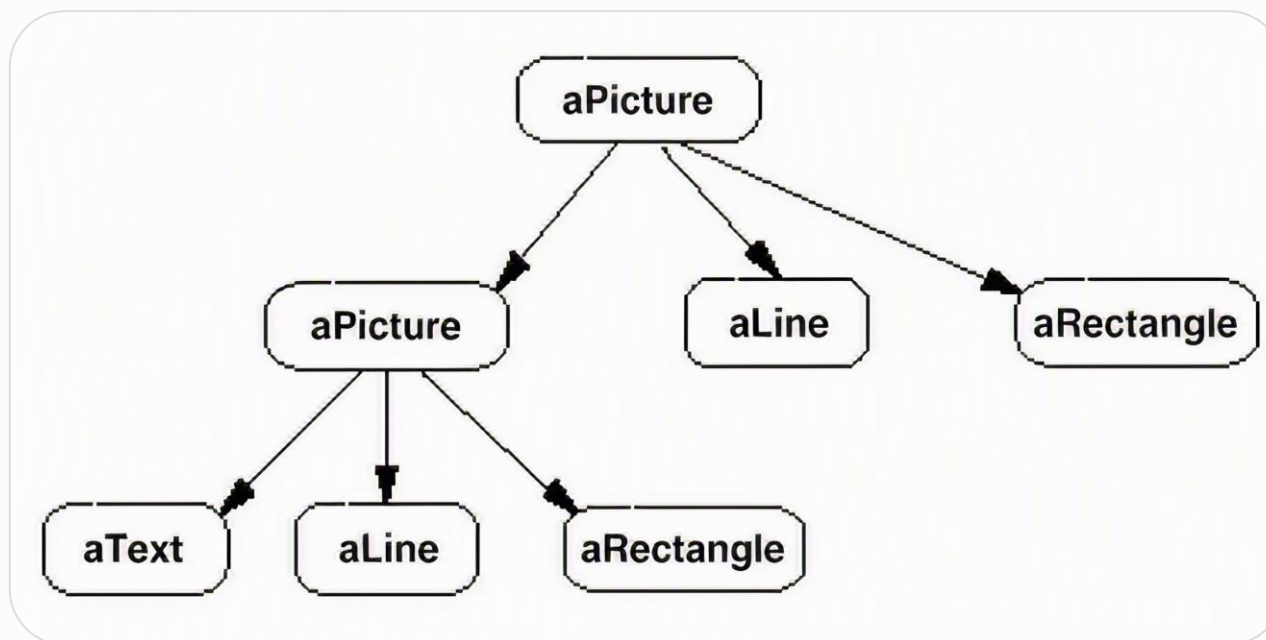
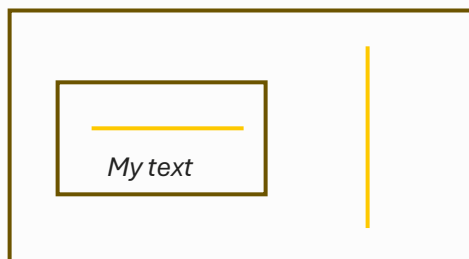


(Shvets, Alexander, 2018)

Poll 6: Examples of Composite



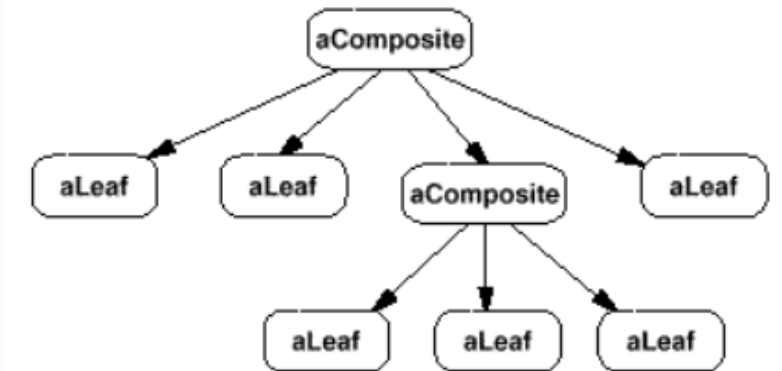
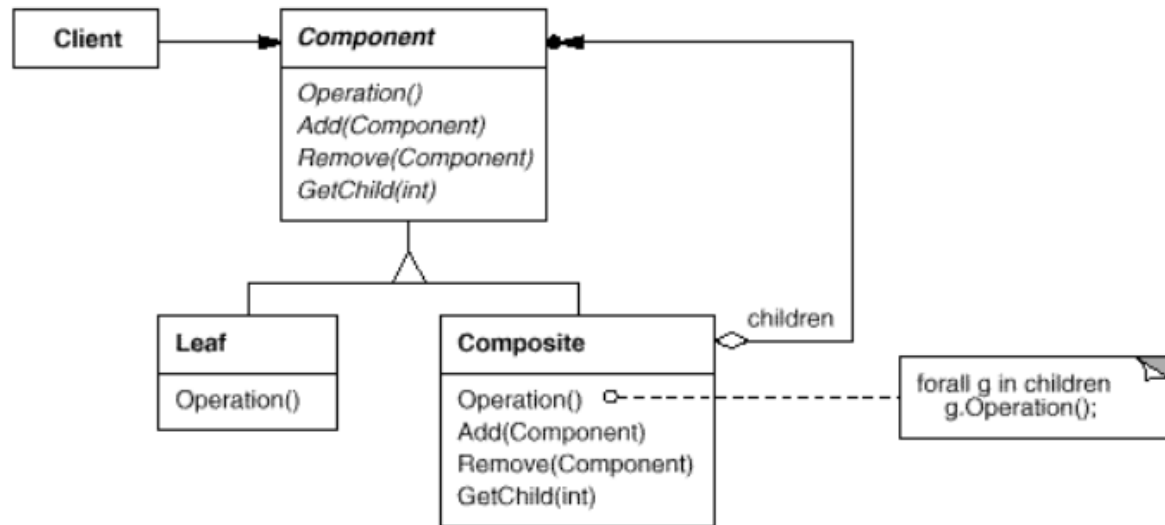
Another example of the Composite Design Pattern



** Image captured from the HTML version of the book published on CD (Gamma et al 1995)*

Composite Design Pattern

- **Applicability**
- Use the Composite pattern when:
 - you want to represent whole-part hierarchies.
 - you want clients to be able to ignore the difference between compositions of objects and individual objects. Clients will treat all objects in the composite structure uniformly.
- **Structure:**



Images captured from the HTML version of the book published on CD (Gamma et al 1995)

Composite Design Pattern

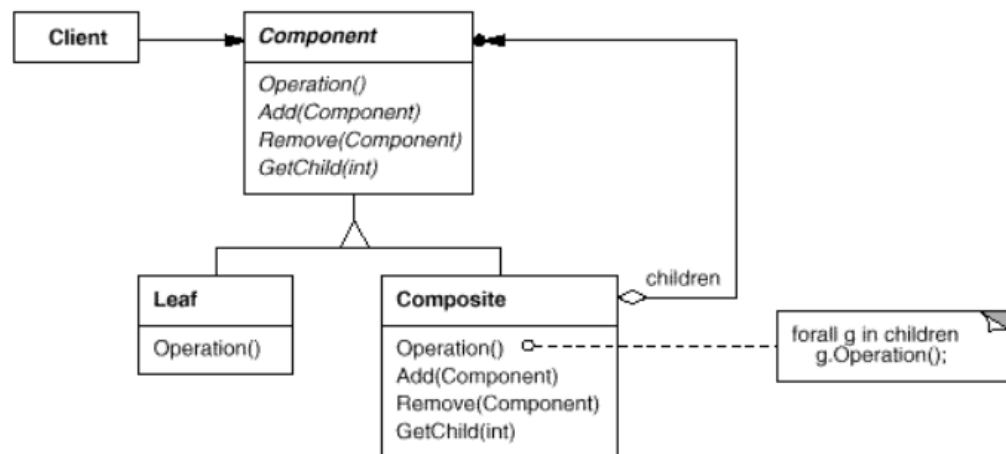
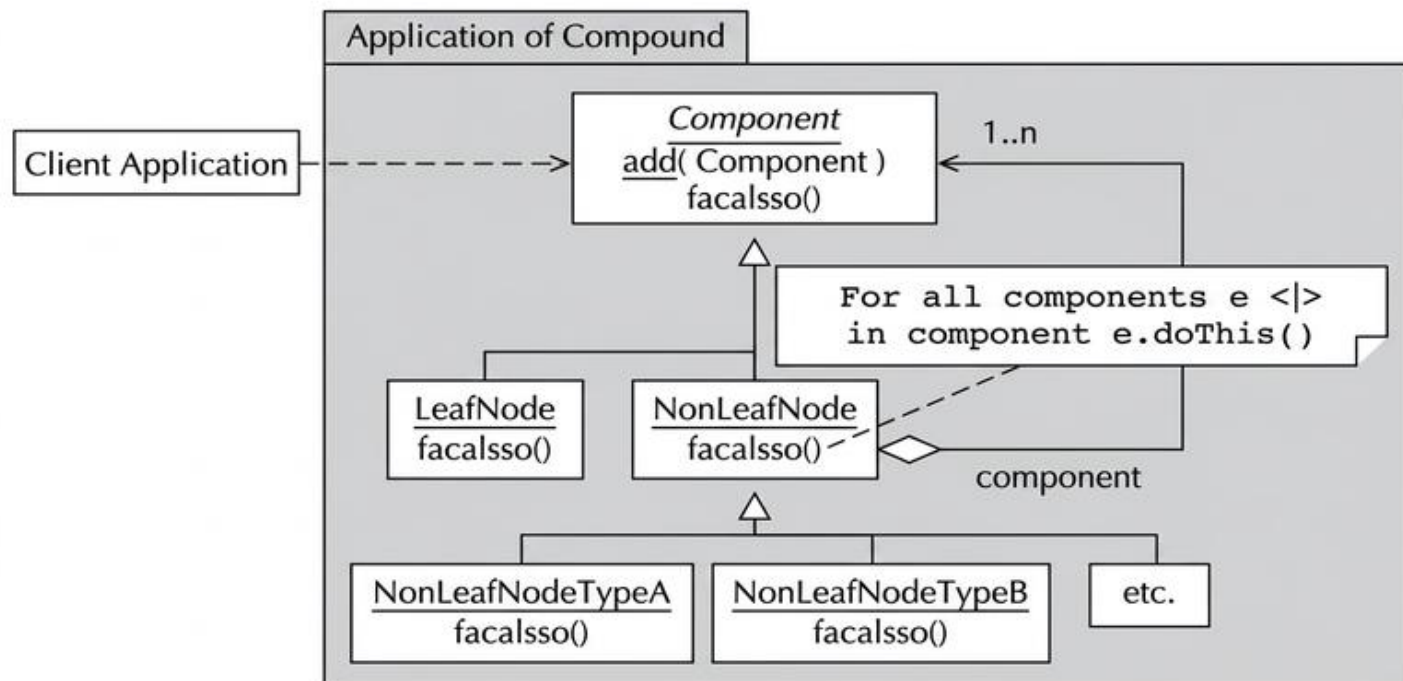


Image captured from the html version of the book published on CD (Gamma et al 1995)



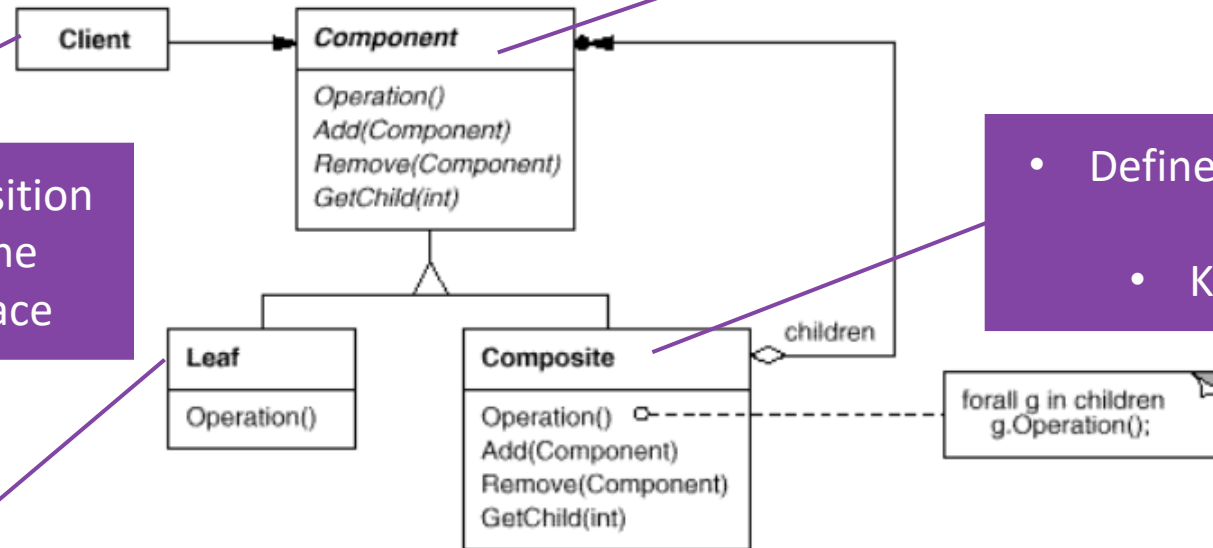
(Braude, Eric, 2005)

Composite Design Pattern

- Participants:

- Declares interface for objects in the composition and for managing child components
- Default behavior goes here, if applicable

- Manipulates composition objects through the Component interface

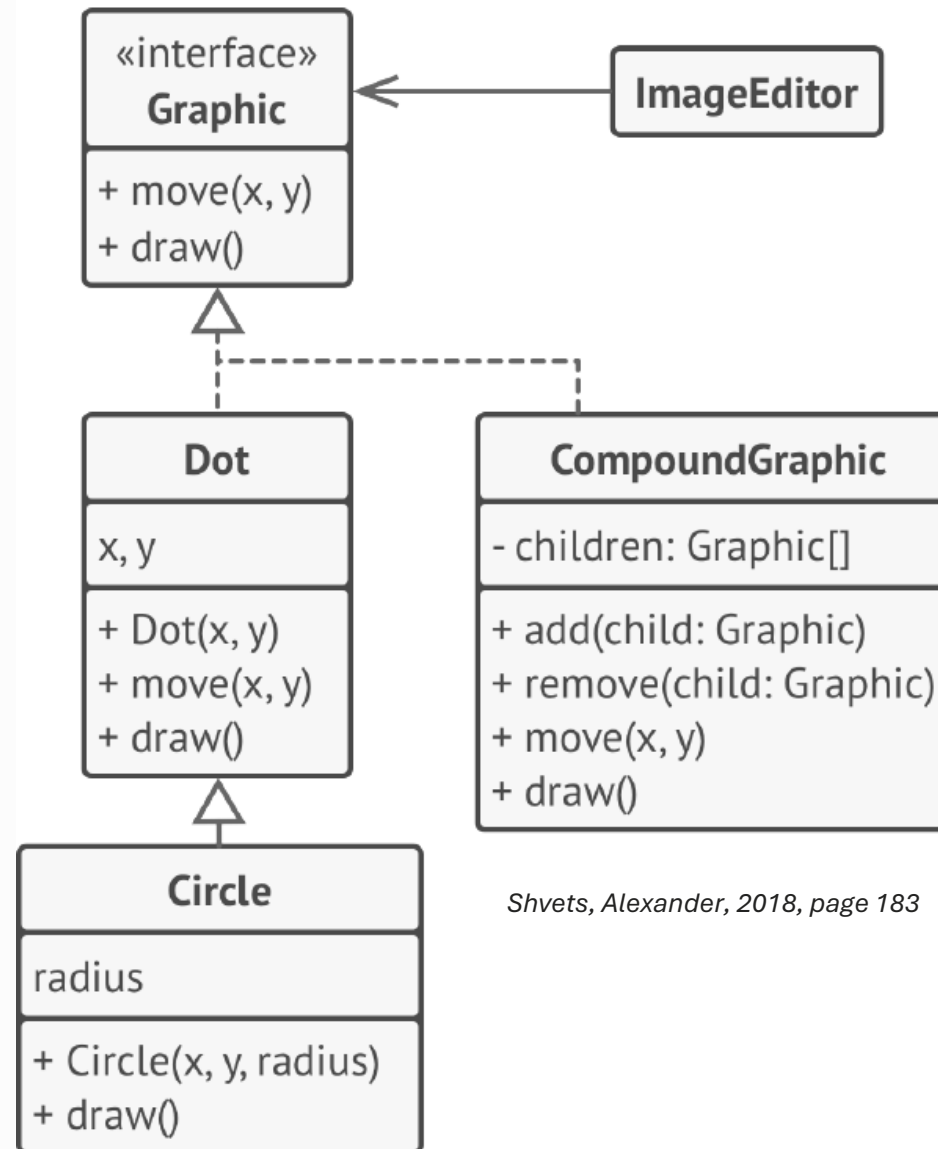


- Defines behavior for objects that have children
- Knows its child components

- Represents leaf objects (that have no children)
- Defines behavior for primitive objects in the composition

Image captured from the html version of the book published on CD (Gamma et al 1995)

Composite Design Pattern code example



Shvets, Alexander, 2018, page 183

Composite Design Pattern code example

```
1  // The component interface declares common operations for both
2  // simple and complex objects of a composition.
3  interface Graphic is
4      method move(x, y)
5      method draw()
6
7  // The leaf class represents end objects of a composition. A
8  // leaf object can't have any sub-objects. Usually, it's leaf
9  // objects that do the actual work, while composite objects only
10 // delegate to their sub-components.
11 class Dot implements Graphic is
12     field x, y
13
14     constructor Dot(x, y) { ... }
15
16     method move(x, y) is
17         this.x += x, this.y += y
18
19     method draw() is
20         // Draw a dot at X and Y.
21
22 // All component classes can extend other components.
23 class Circle extends Dot is
24     field radius
25
26
```

Shvets, Alexander, 2018, page 184

Composite Design

Pattern code example

```
27     constructor Circle(x, y, radius) { ... }
28
29     method draw() is
30         // Draw a circle at X and Y with radius R.
31
32     // The composite class represents complex components that may
33     // have children. Composite objects usually delegate the actual
34     // work to their children and then "sum up" the result.
35     class CompoundGraphic implements Graphic is
36         field children: array of Graphic
37
38         // A composite object can add or remove other components
39         // (both simple or complex) to or from its child list.
40         method add(child: Graphic) is
41             // Add a child to the array of children.
42
43         method remove(child: Graphic) is
44             // Remove a child from the array of children.
45
46         method move(x, y) is
47             foreach (child in children) do
48                 child.move(x, y)
49
50         // A composite executes its primary logic in a particular
51         // way. It traverses recursively through all its children,
52         // collecting and summing up their results. Since the
53         // composite's children pass these calls to their own
54         // children and so forth, the whole object tree is traversed
55         // as a result.
56         method draw() is
57             // 1. For each child component:
58             //     - Draw the component.
```

Shvets, Alexander, 2018, page 185

Composite Design

Pattern code example

```
59      //      - Update the bounding rectangle.
60      // 2. Draw a dashed rectangle using the bounding
61      // coordinates.
62
63
64      // The client code works with all the components via their base
65      // interface. This way the client code can support simple leaf
66      // components as well as complex composites.
67      class ImageEditor is
68          method load() is
69              all = new CompoundGraphic()
70              all.add(new Dot(1, 2))
71              all.add(new Circle(5, 3, 10))
72              // ...
73
74      // Combine selected components into one complex composite
75      // component.
76      method groupSelected(components: array of Graphic) is
77          group = new CompoundGraphic()
78          group.add(components)
79          all.remove(components)
80          all.add(group)
81          // All components will be drawn.
82          all.draw()
```

Shvets, Alexander, 2018, page 186

Consequences of using the Composite

- Primitive objects can be composed into more complex objects, which in turn can be composed, and so on recursively:
 - wherever client code expects a primitive object, it can also receive a composite object.
- Makes the client simple.
 - Clients do not need to know if they are dealing with a leaf or a composite component
 - avoids having to write "case" style functions
- Makes it easier to add new types of components: clients don't need to be changed for new Component classes.
- Negative points:
 - the design can become too generic.
 - it becomes more difficult to restrict the components of a composite. For example, if your composite is required to have only certain components. You cannot rely on the programming language's type system to enforce these restrictions, but will have to use runtime checks.

Patterns related to the Composite

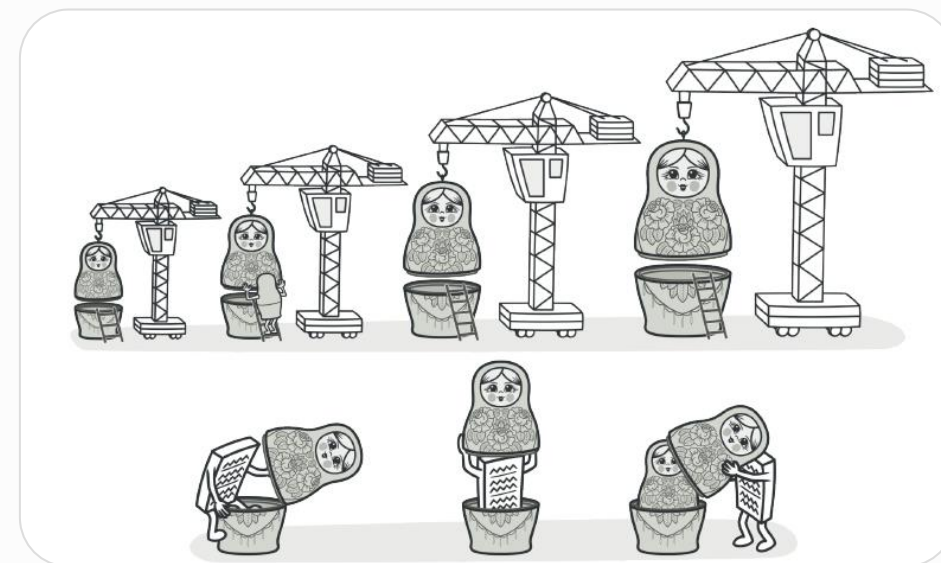
- Chain of Responsibility
- Decorator
- Flyweight
- Iterator
- Visitor

Resuming poll 6

- Can you draw a class diagram applying the Composite Pattern in another example?

Decorator Design Pattern

- **Classification:** Structural Pattern/Object Scope
- **Objective:** Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to inheritance for extending functionality.
- **Application:** Use Decorator to:
 - add responsibilities to individual objects dynamically and transparently, i.e., without affecting other objects.
 - Allow responsibilities to be removed.
 - Avoid a large number of independent extensions producing a subclass explosion to support every combination.



(Shvets, Alexander, 2018)

Decorator Design Pattern

- Structure:

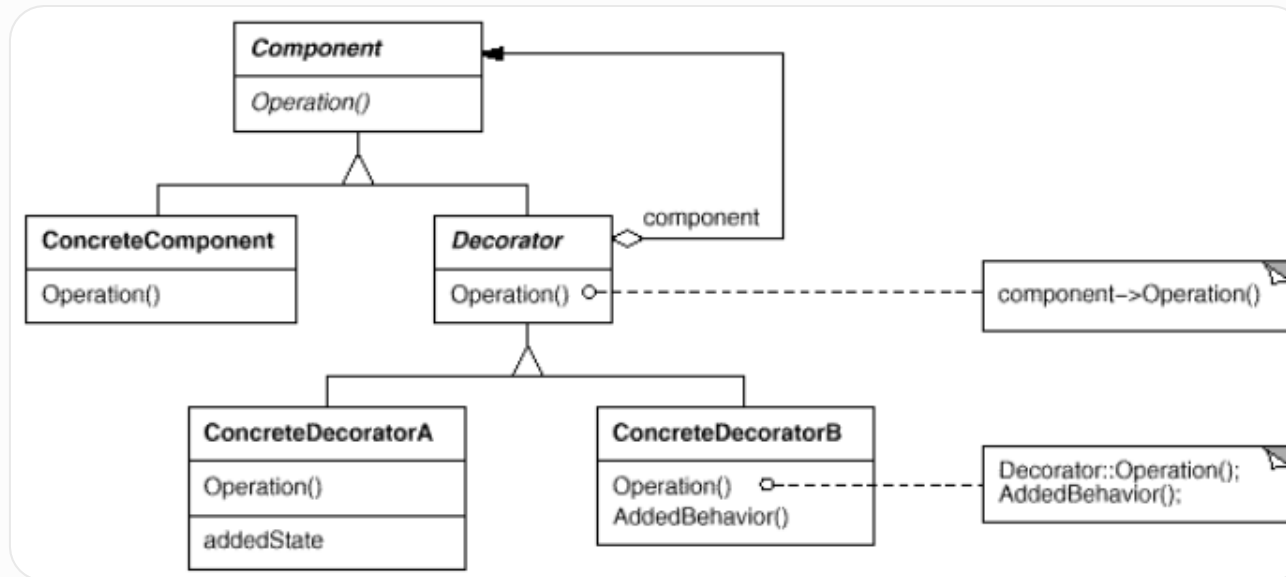
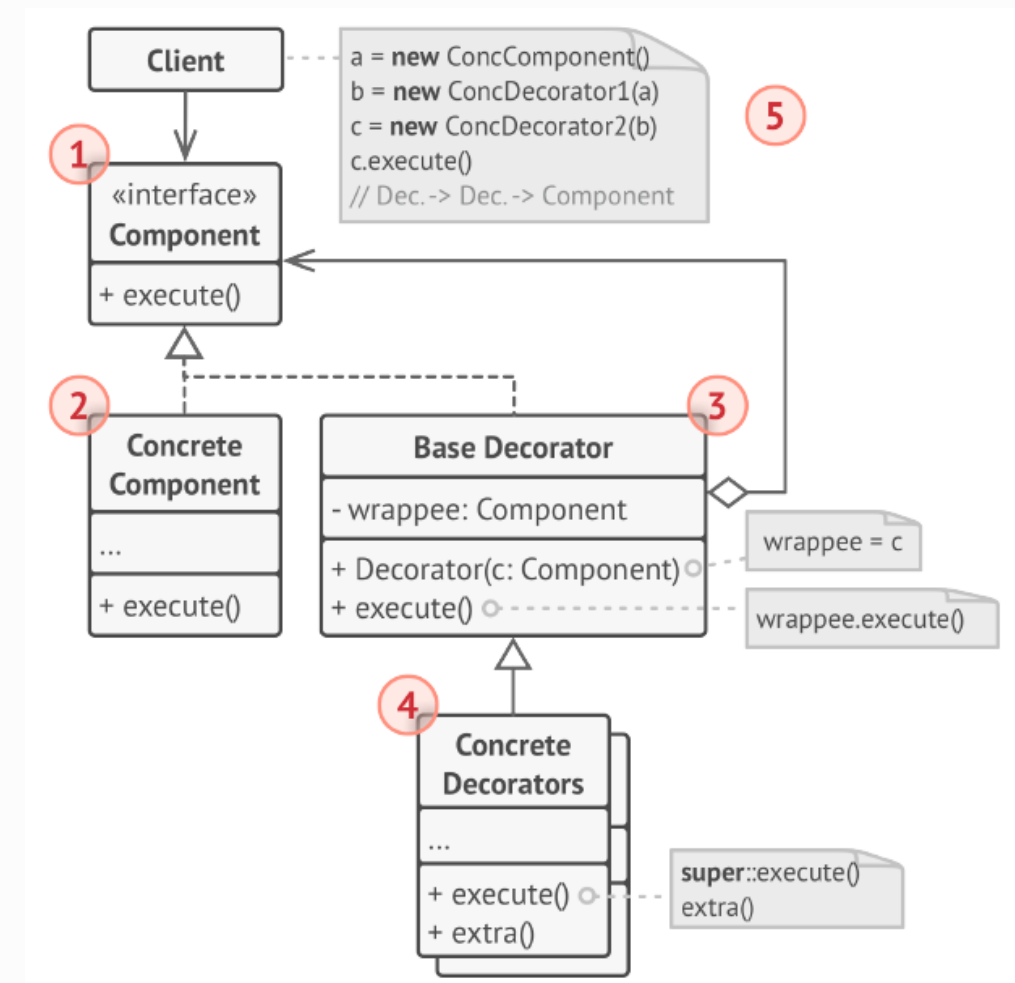
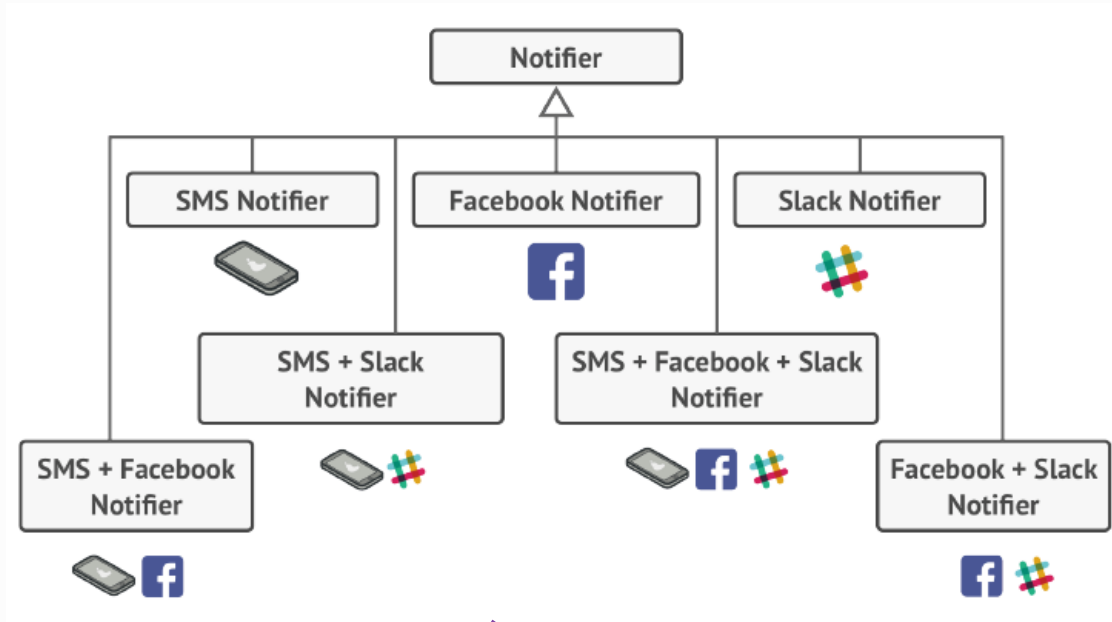


Image captured from the html version of the book published on CD (Gamma et al 1995)

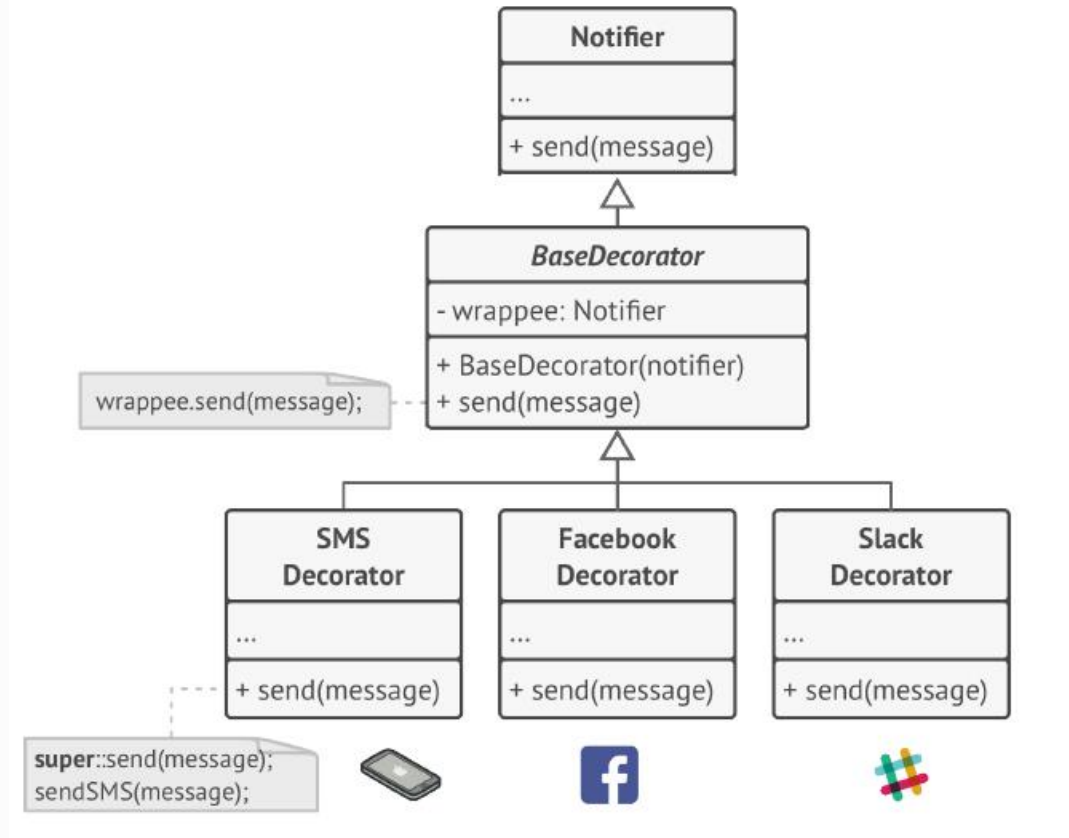


(Shvets, Alexander, 2018)

Decorator Design Pattern



- Without using the Decorator

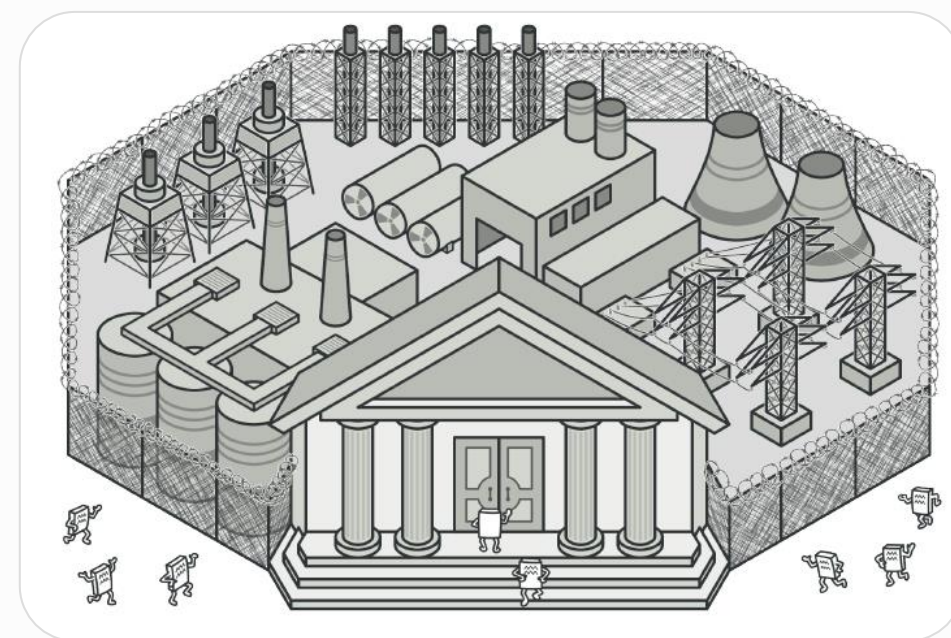


- Using the Decorator

(Shvets, Alexander, 2018)

Facade Design Pattern

- **Classification:** Structural pattern/Object scope
- **Objective:** Provide a unified interface to a set of interfaces in a subsystem. Facades define a higher-level interface that makes the subsystem easier to use.
- **Application:** you need to integrate your application or system with a sophisticated library that has dozens of features, but you only need a part of the functionality



(Shvets, Alexander, 2018)

Facade Design Pattern

- Structure:

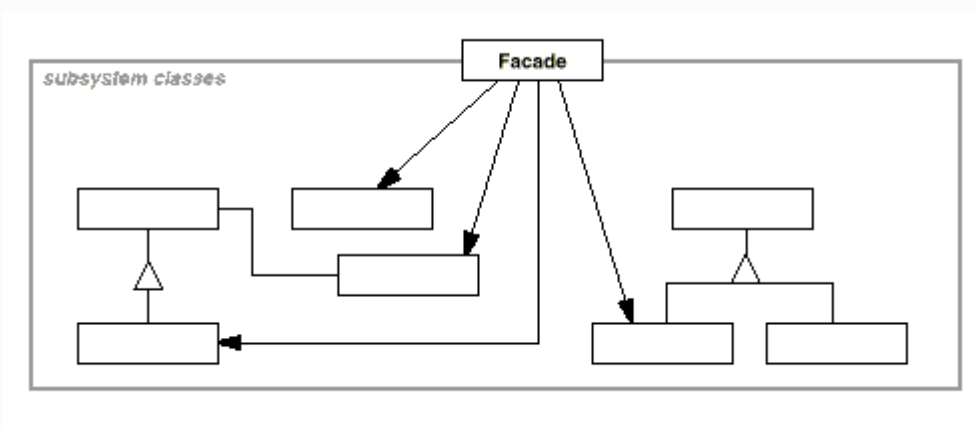
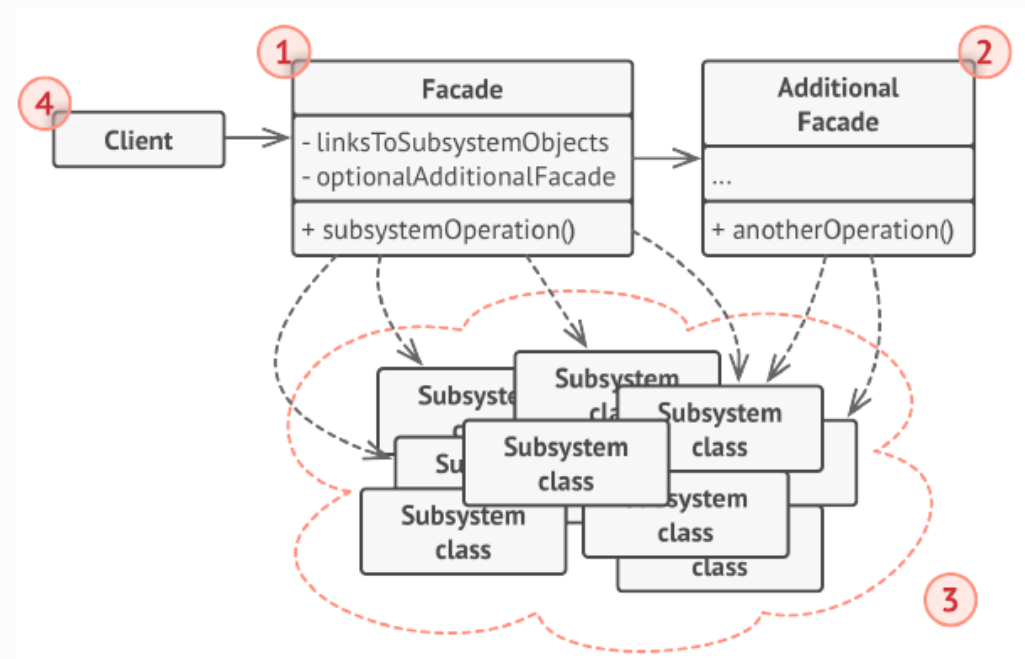


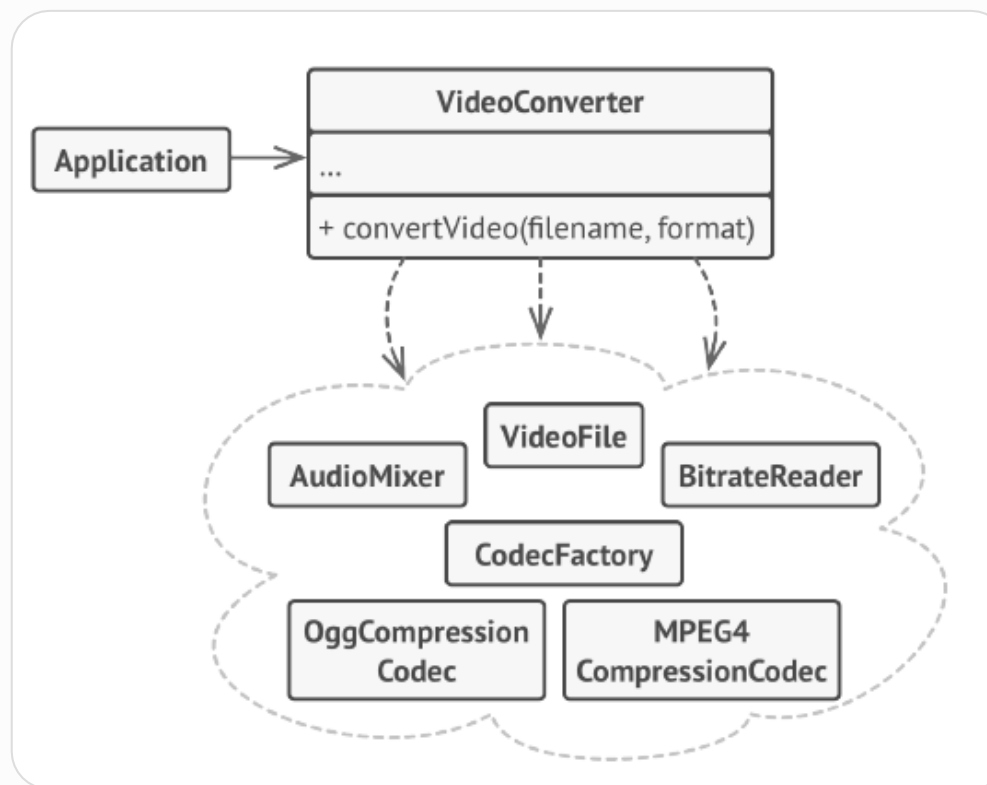
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Facade Design Pattern

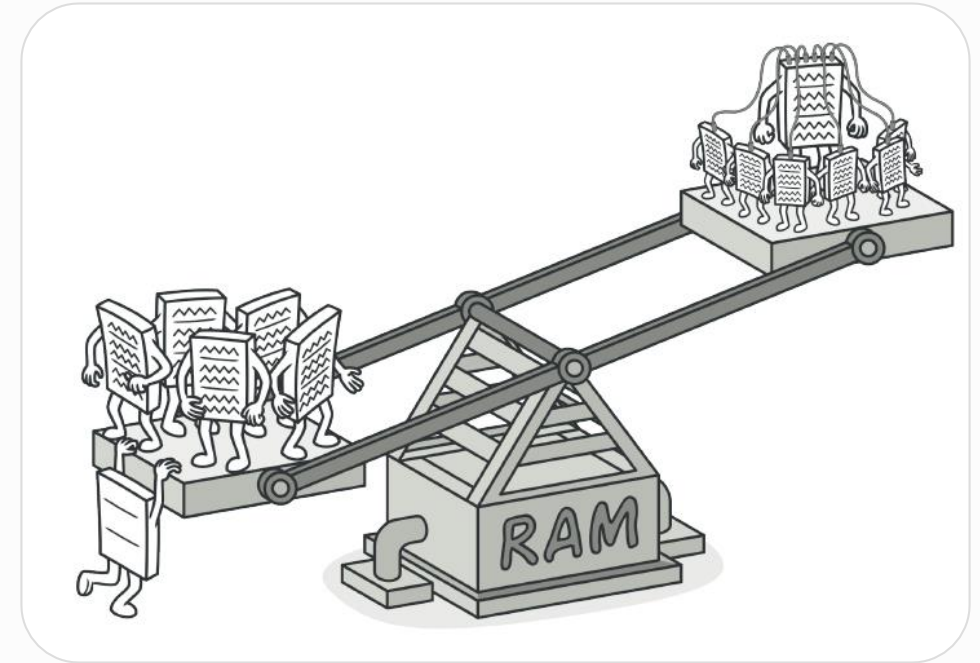
- Example:



(Shvets, Alexander, 2018)

Flyweight Design Pattern

- **Classification:** Structural pattern/Object scope
- **Objective:** Use sharing to efficiently support a large number of fine-grained objects. This pattern allows, for example, that you fit more objects into the available RAM by sharing common parts of state among multiple objects instead of keeping all data in each object.
- **Application:** Use the Flyweight pattern only if your program needs to support a large number of similar objects that are overloading the available RAM. Objects must contain duplicate states that can be extracted and shared among multiple objects.



(Shvets, Alexander, 2018)

Flyweight Design Pattern

- Structure:

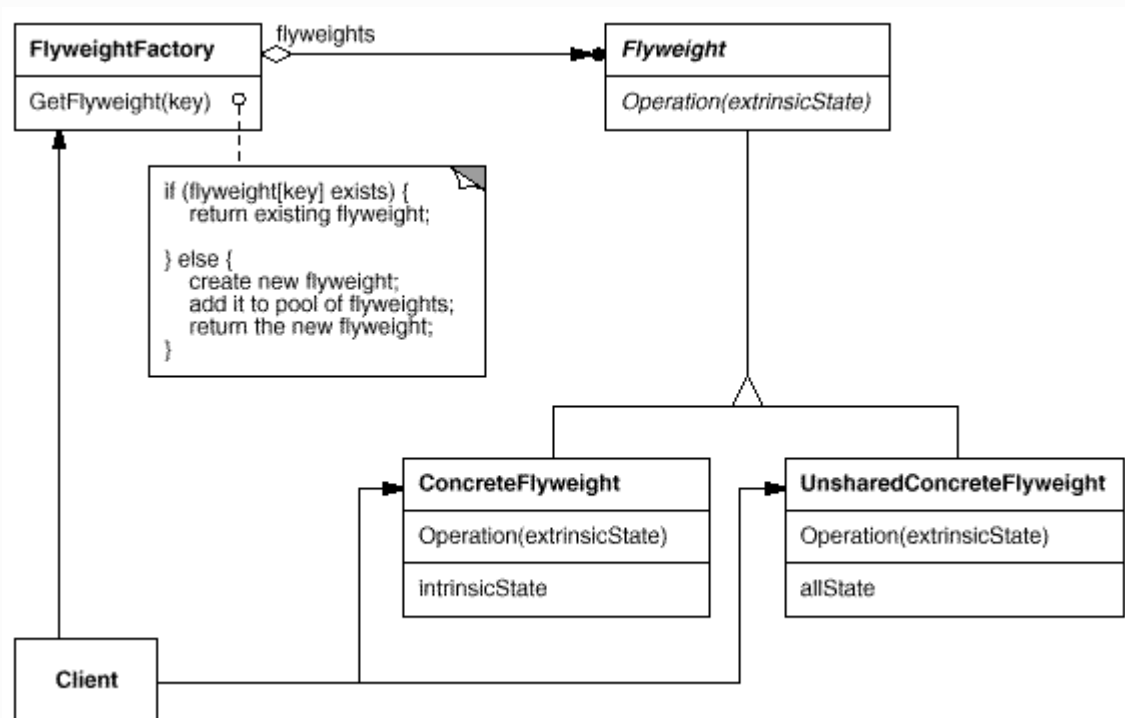
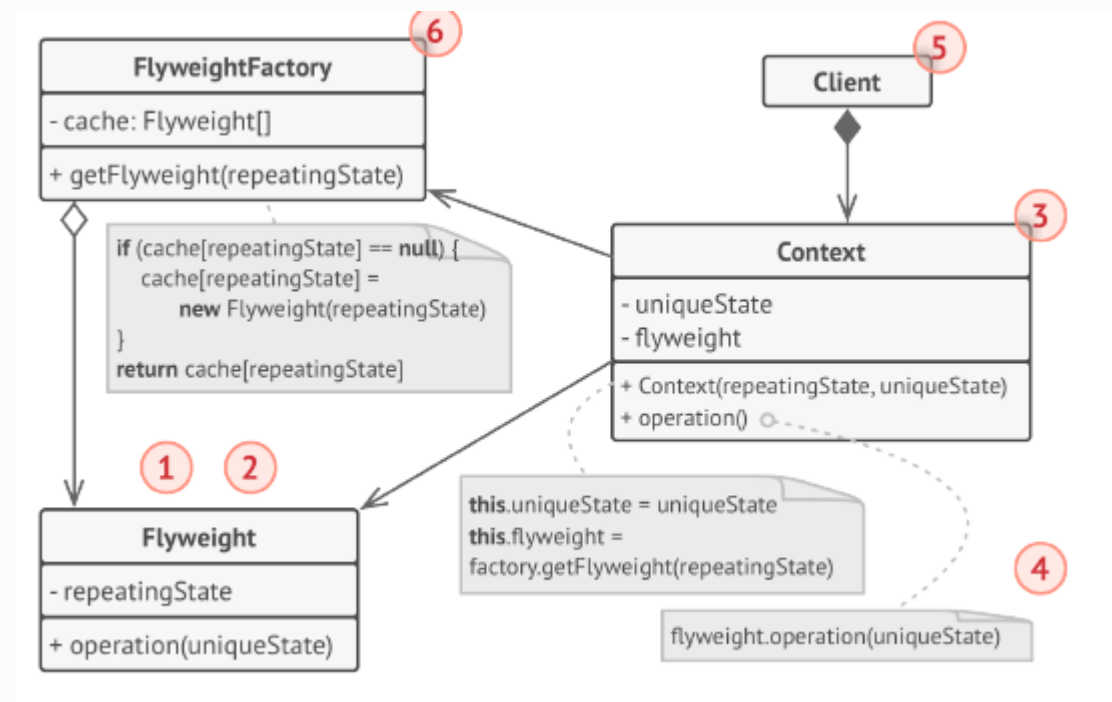


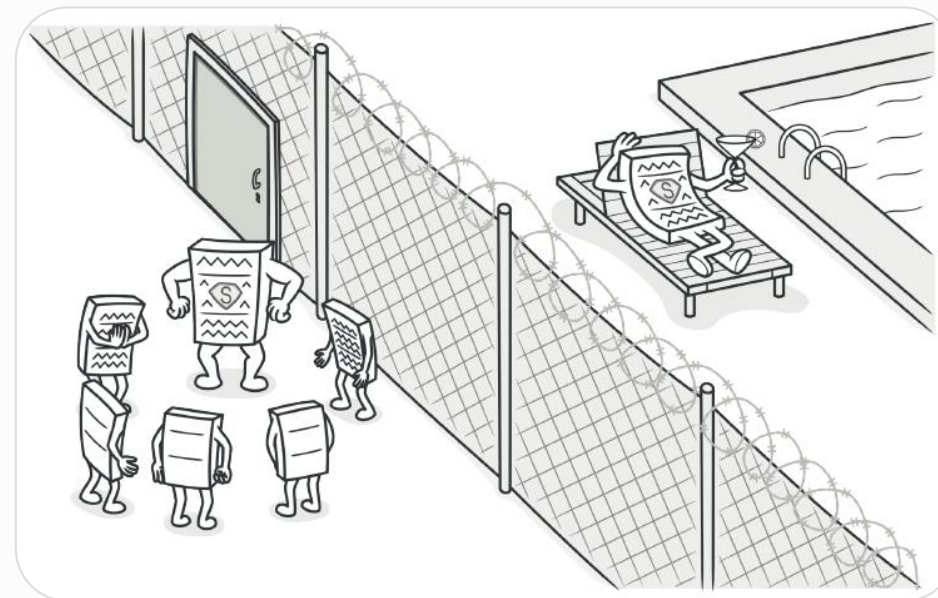
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Proxy Design Pattern (from English Proxy)

- **Classification:** Structural pattern/Object scope
- **Objective:** Provide a substitute or placeholder for another object to control access to it.
- **Motivation:** We may want to control access to an object, for example, to defer the total cost of its creation and initialization until we actually need to use it. The cost of object creation is generally due to the fact that many attributes need to be initialized, in addition to dependencies with other objects that compose the object.



(Shvets, Alexander, 2018)

Proxy Design Pattern

- **Applicability**
- Uses of the Proxy pattern:
 - *Remote proxy*: it is necessary to reference an object in a different address space,
 - *Virtual proxy*: it is desired to create objects on demand (defer the object's materialization until it is actually needed), or “*lazy initialization*”
 - *Protection proxy*: it is desired to maintain greater control over the original object (access control for example),
 - *Smart reference*: it is desired to maintain a “smart” reference to the object (the pointer to the object performs some additional behavior whenever the object is accessed).

Poll 7: Proxy Usage



Proxy Design Pattern

- **Solution**

- Provide a substitute, or proxy, or even a placeholder for the object, with the specific purpose of controlling access to the object.
- The Proxy maintains a reference to the Real Subject and has the same interface so that it can be substituted for the Real Subject.
- Generally, it is the Proxy that creates the Real Subject, when necessary.
- The Proxy delegates requests to the Real Subject only when necessary. This depends on the type of Proxy, as several types are described in the original version of the Proxy pattern (Gamma et al. 1995).

Proxy Design Pattern

- Structure:

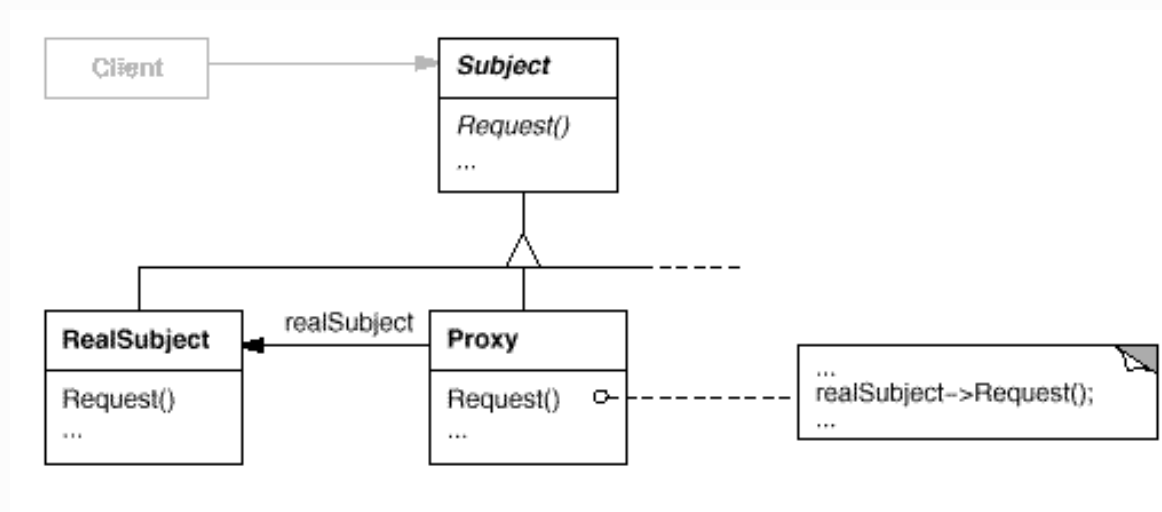
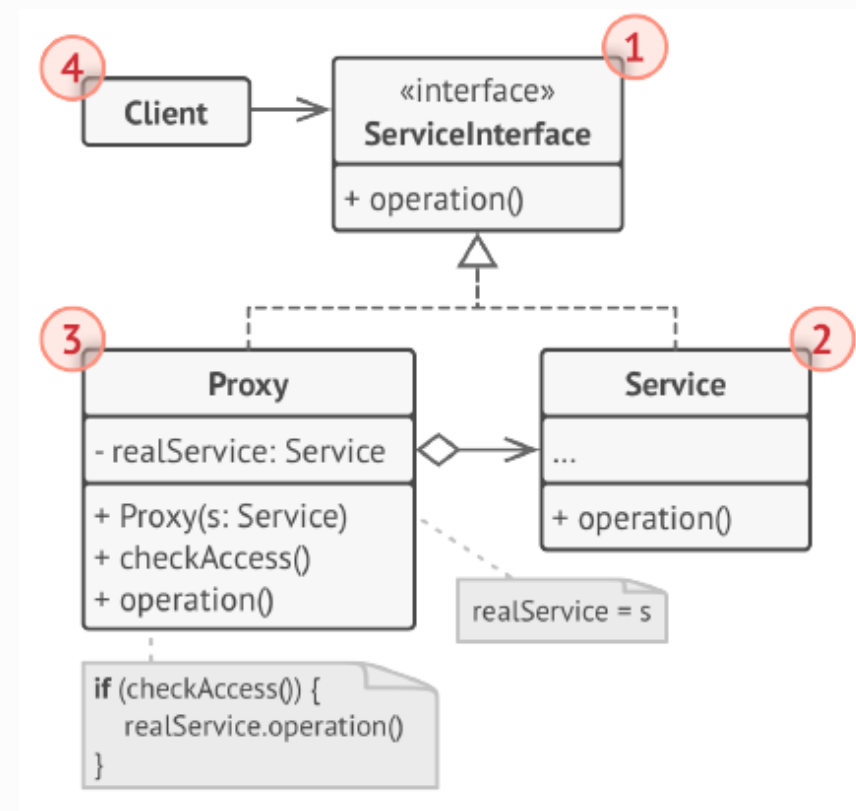


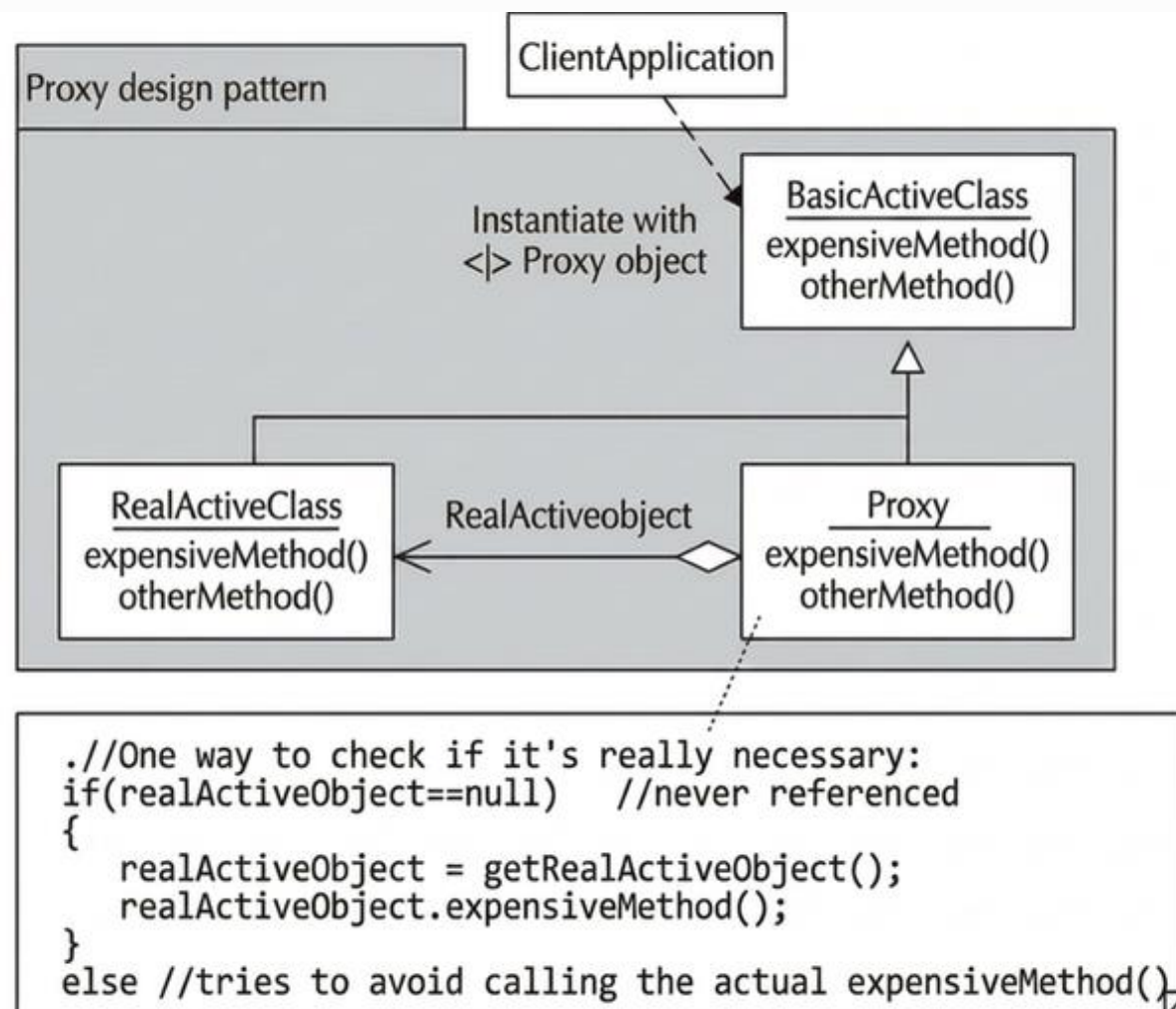
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Proxy Design Pattern

- Structure:



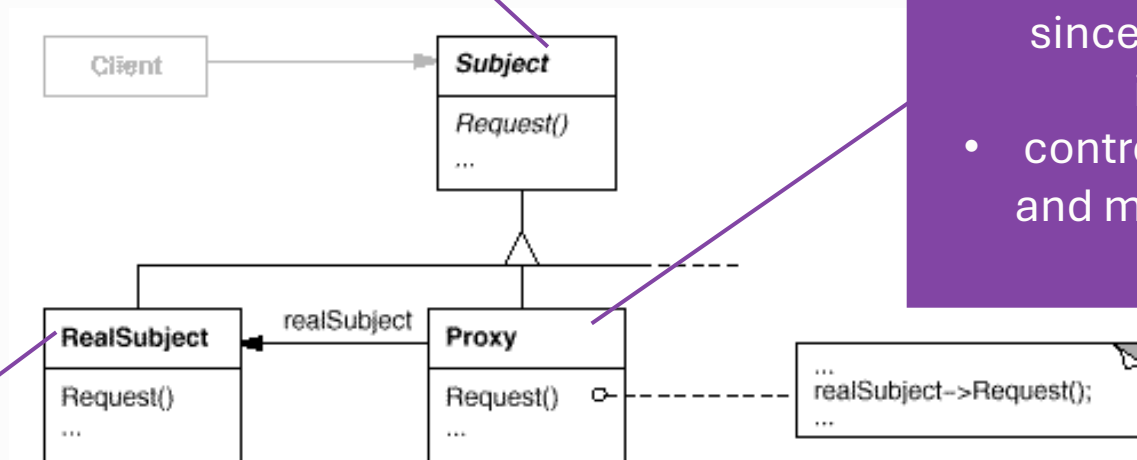
(Braude, Erick, 2005)

Proxy Design Pattern

- Participants:

Defines the common interface for RealSubject and Proxy, so that the Proxy can be used in place of the RealSubject

- maintains a reference that allows the Proxy to access the RealSubject, since the RealSubject and Subject interfaces are the same.
- controls access to the RealSubject and may be responsible for creating and deleting it.



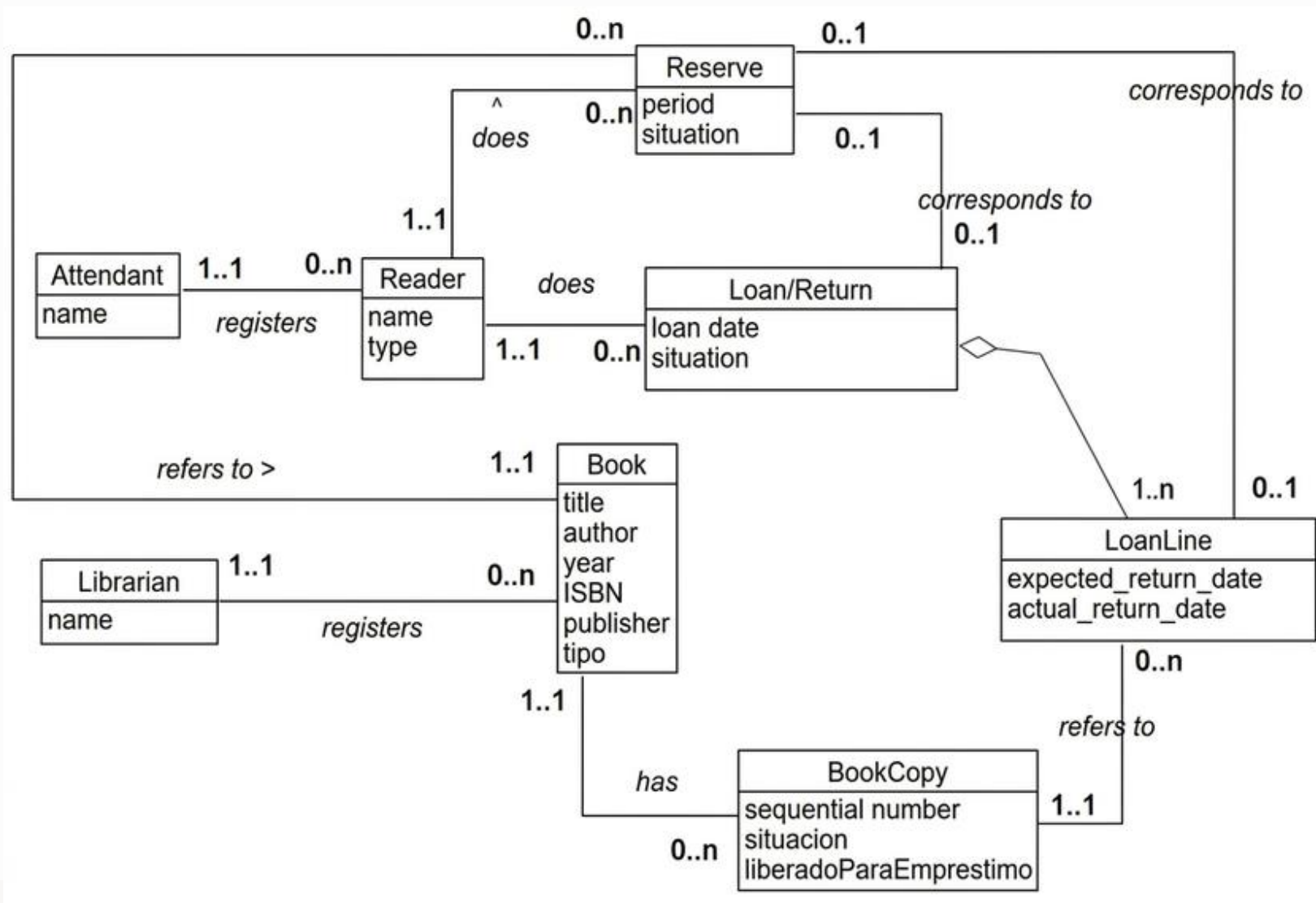
Defines the RealSubject that the proxy represents

Image captured from the html version of the book published on CD (Gamma et al 1995)

Proxy Design Pattern

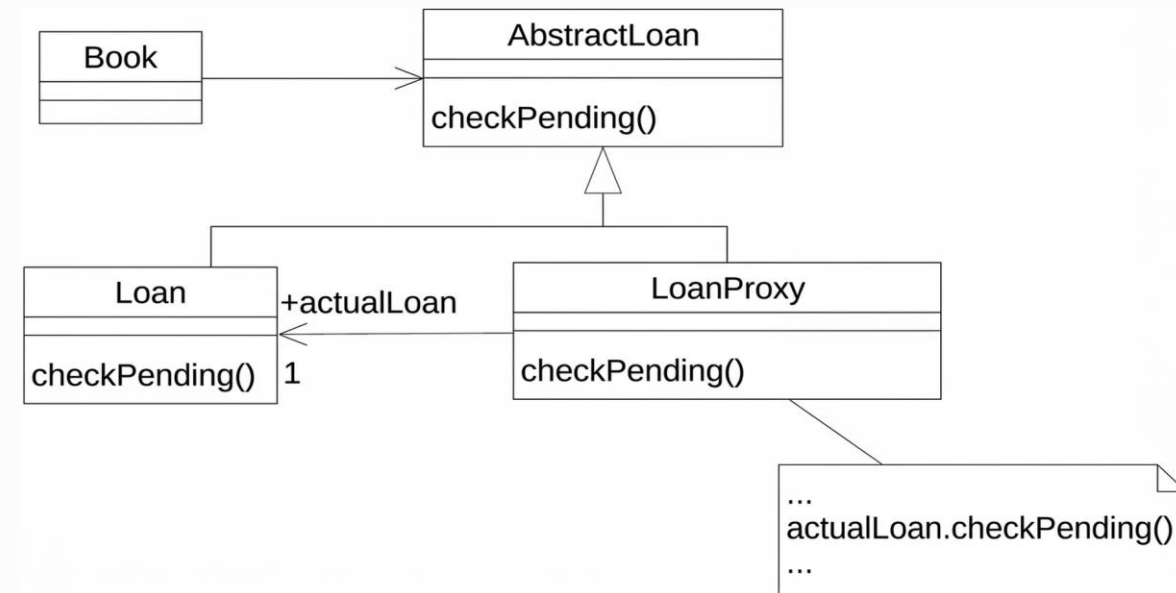
- **Example: object materialization and dematerialization**
 - Materialization: bringing a record from a table in the form of an object
 - Dematerialization: persisting an object as a record in a table
- In a library system, when materializing a reader, we can at the same time materialize the objects referring to all that reader's loans.
 - However, the loan objects might be materialized uselessly, as the system will probably not use information about these objects, wasting time and space.

Proxy Design Pattern



Proxy Design Pattern

- The **Virtual Proxy** variant can be used to solve this problem
- Which objects associated with object X should be retrieved every time an object X is retrieved?
- For objects that will not need to be retrieved, a proxy is created.
 - Example: when materializing a Book, should its associated loans also be materialized?
 - Answer: no



		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template Method
	Object	• Abstract Factory • Builder • Prototype • Singleton	• Adapter • Bridge • Composite • Decorator • Facade • Flyweight • Proxy	• Chain of Responsibility • Command • Iterator • Mediator • Memento • Observer • State • Strategy • Visitor

“behavioral” category PATTERNS

Responsible for characterizing the way classes and objects interact with each other and distribute responsibilities

Interpreter Design Pattern

- **Classification:** Behavioral pattern/Class scope
- **Intent:** Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.
- **Structure:**

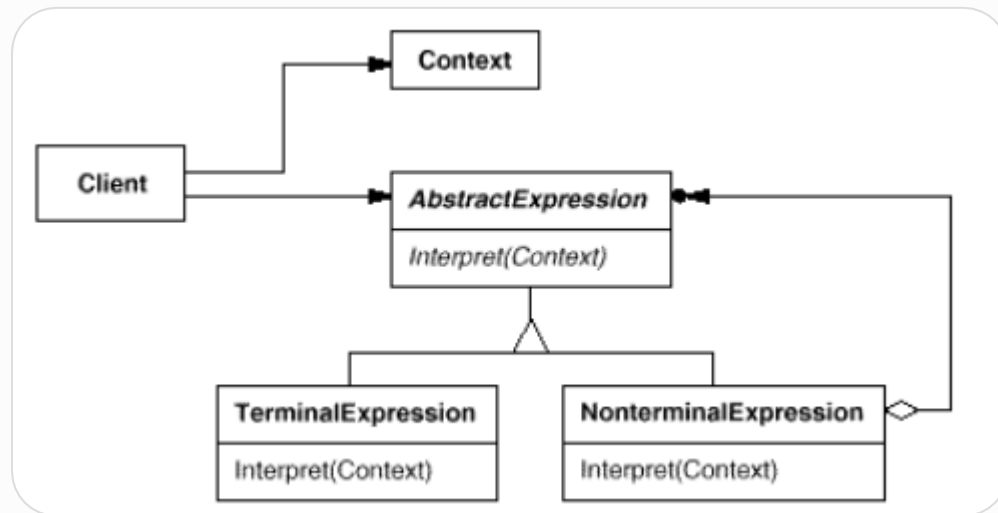
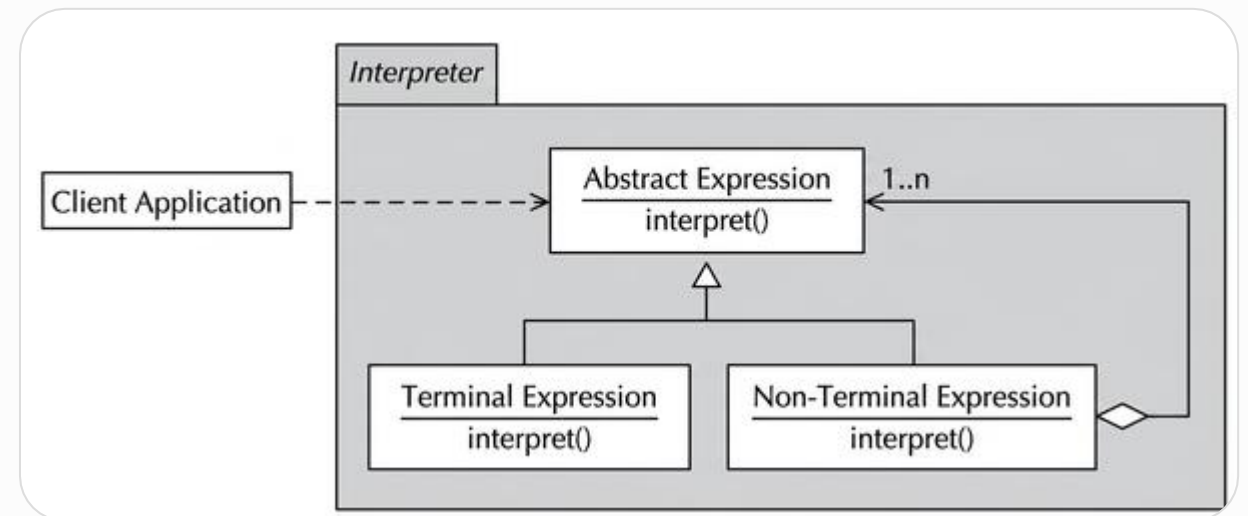


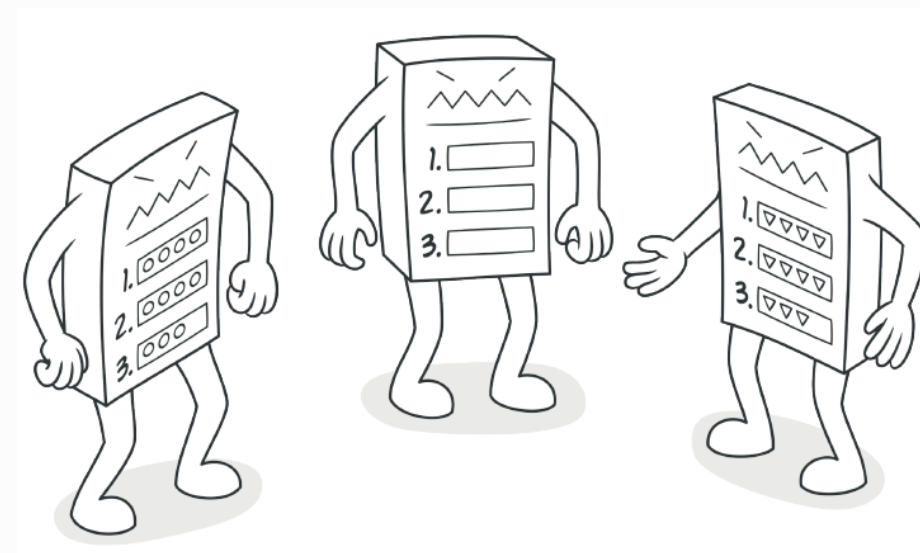
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Braude, Erick, 2005)

Template Method Design Pattern

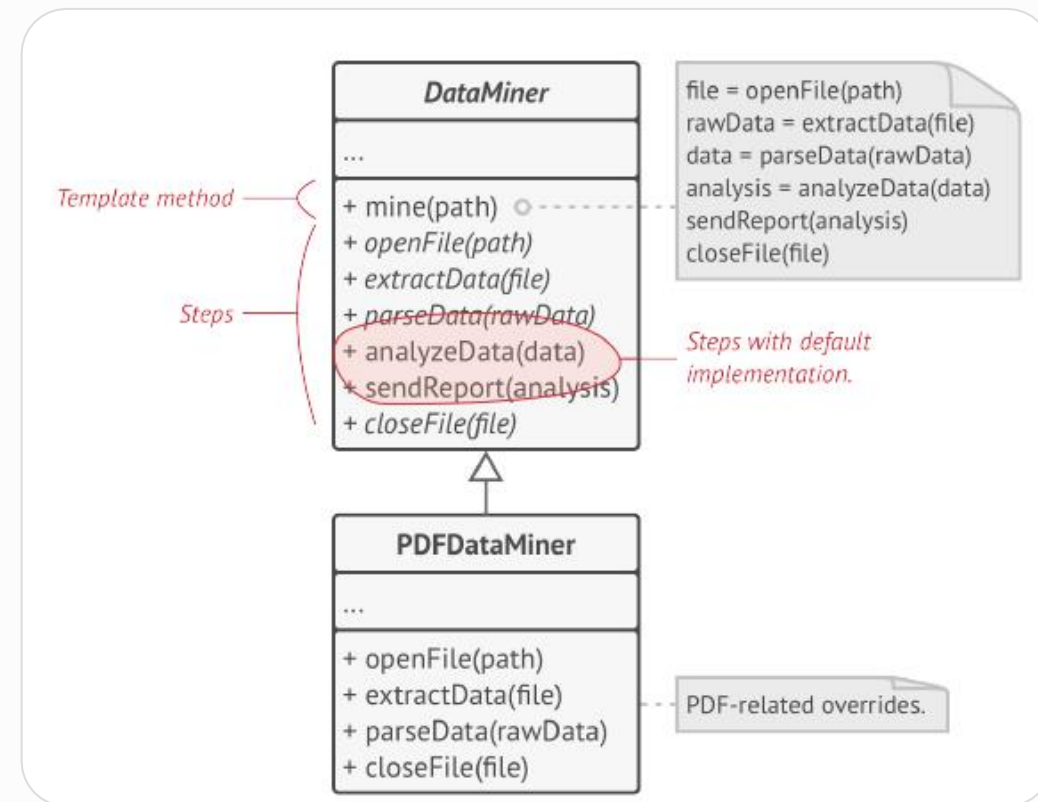
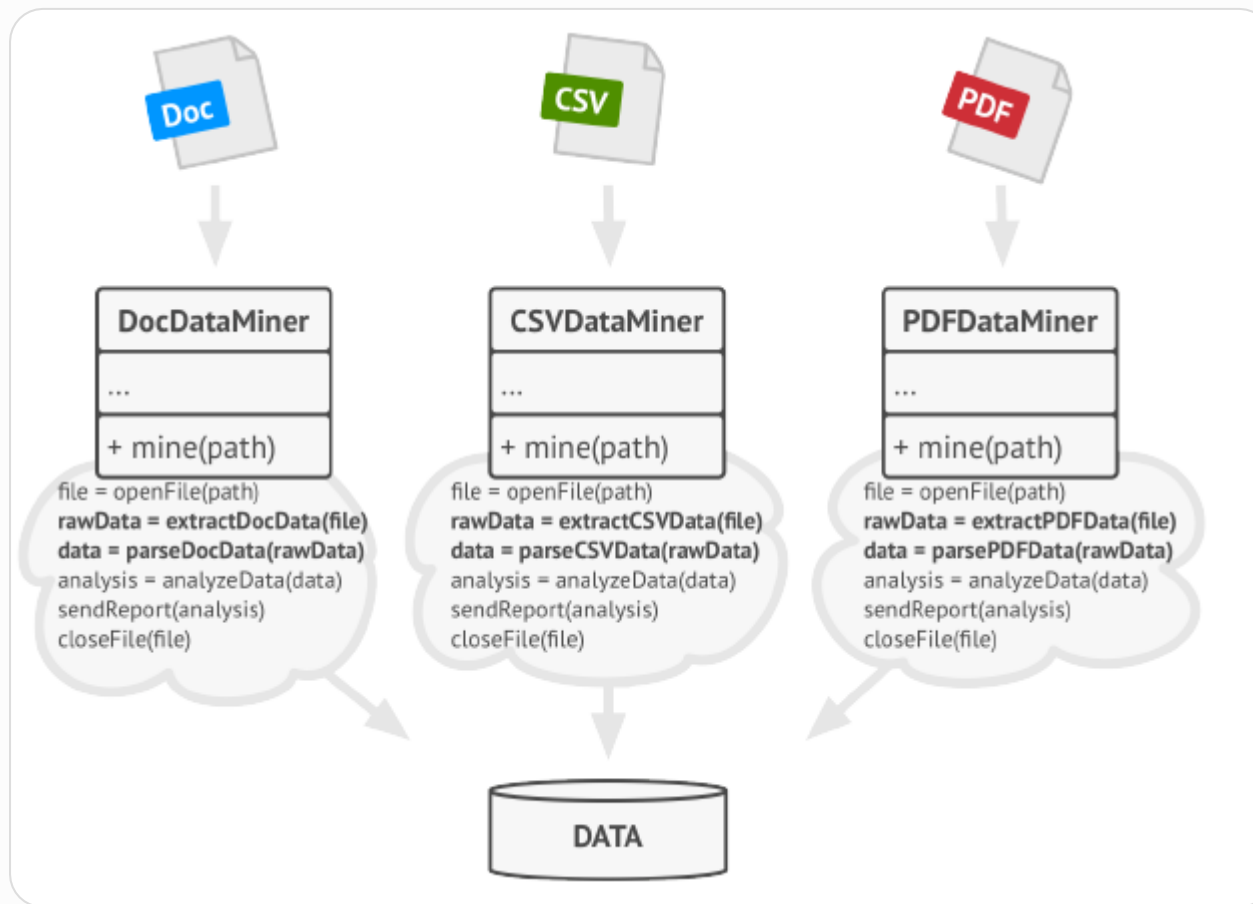
- **Classification:** Behavioral pattern/Class scope
- **Objective:** Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. The Template Method allows subclasses to redefine certain steps of an algorithm without changing the algorithm's structure.
 - The idea of the Template Method is to define a template method in a superclass, which contains the algorithm's skeleton, with its variable and invariable parts. This method invokes other methods, some of which are operations that can be defined in a subclass.
 - Thus, subclasses can redefine the methods that vary, adding specific behavior to the variable points.



(Shvets, Alexander, 2018)

Template Method Design Pattern

Applying the pattern ...



(Shvets, Alexander, 2018)

Template Method Design Pattern

- Structure:

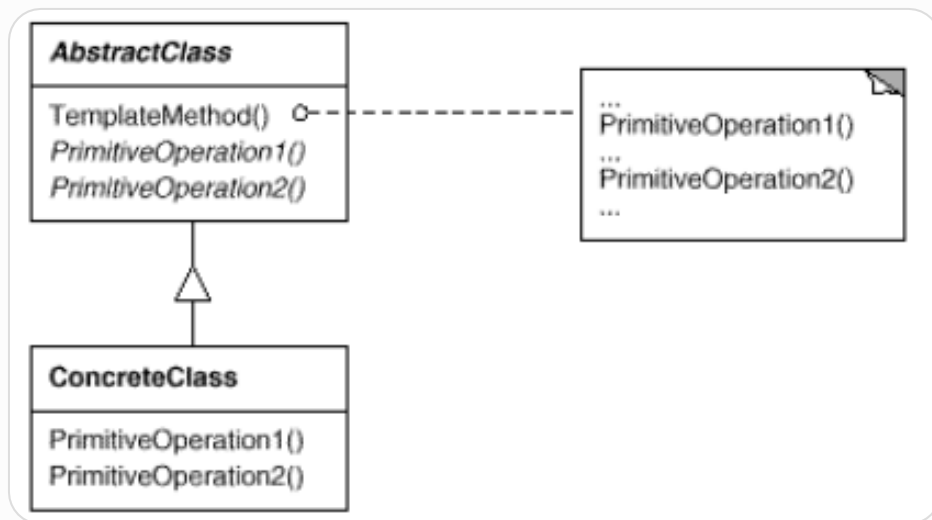
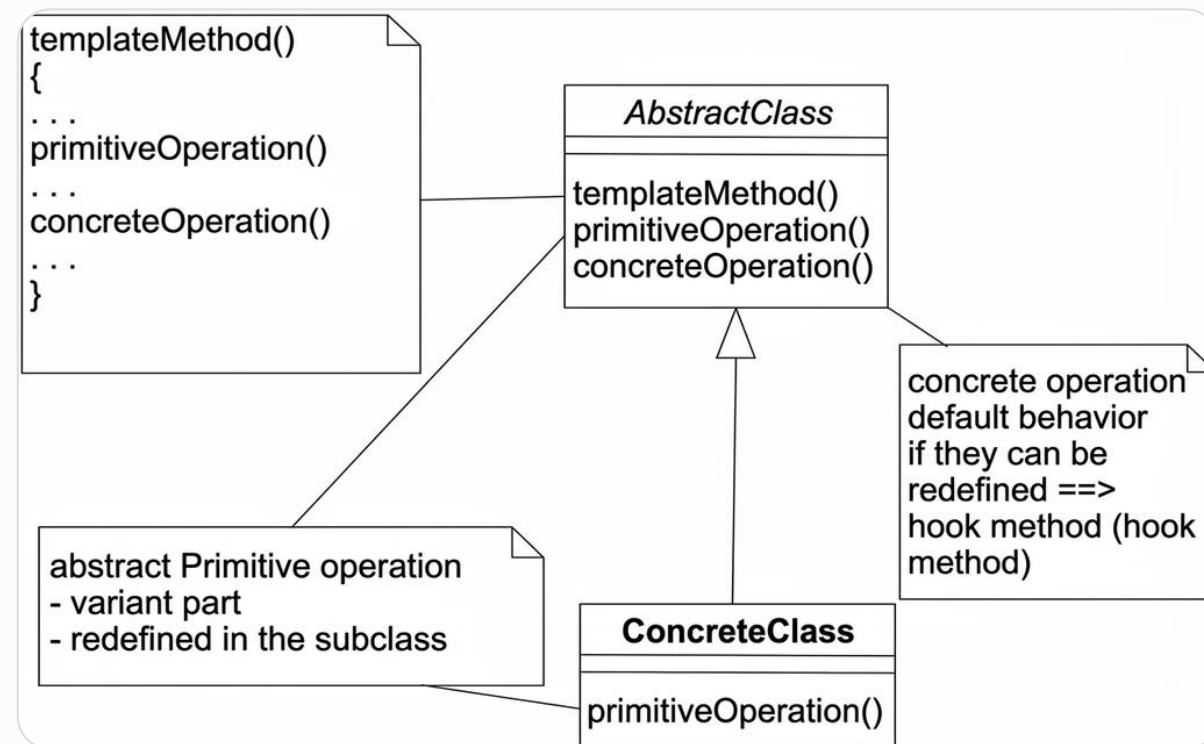


Image captured from the html version of the book published on CD (Gamma et al 1995)



(Braude, 2005)

Template Method Design Pattern

- Participants:

- defines abstract primitive operations that concrete subclasses implement as steps of an algorithm.
- implements a template method that defines the skeleton of an algorithm. It calls primitive operations, as well as operations defined in the abstract class or even in other objects.

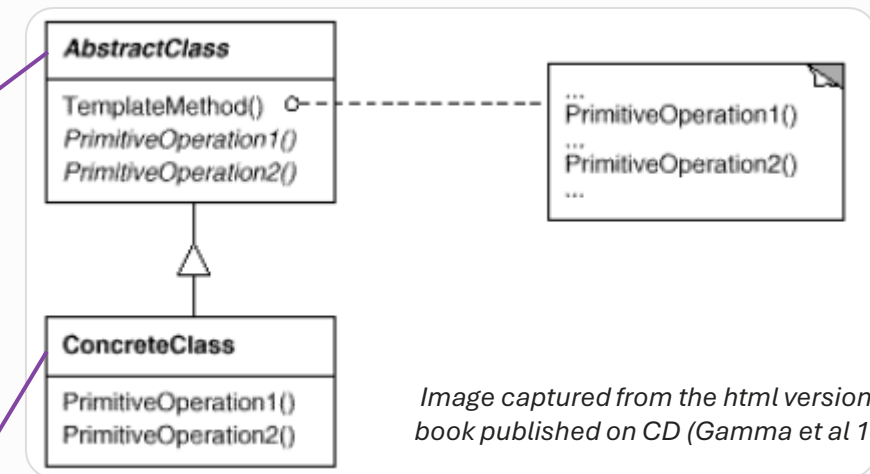


Image captured from the html version of the book published on CD (Gamma et al 1995)

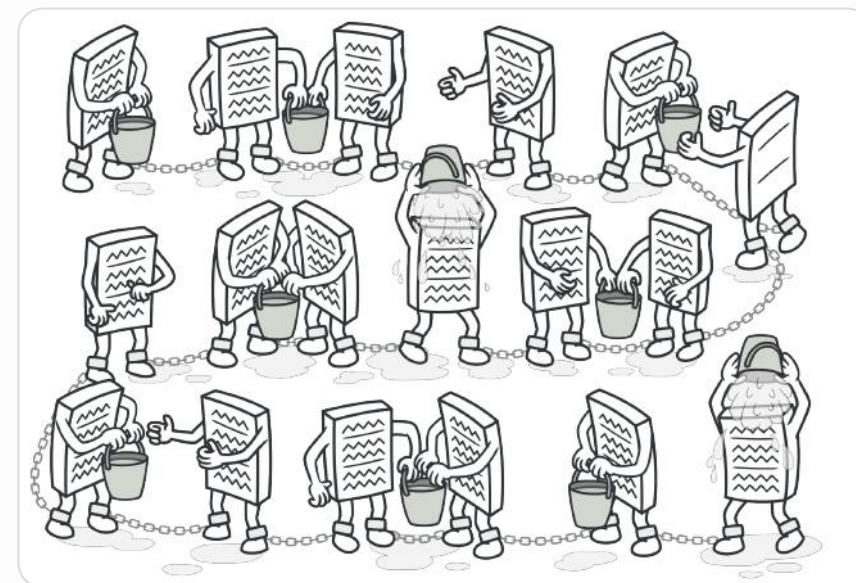
- implements the primitive operations to perform subclass-specific steps of the algorithm.

Poll 8: Related pattern?



Chain of Responsibilities Design Pattern

- **Classification:** Behavioral pattern/Object scope
- **Objective:** Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.
- **Applicability:** use the Chain of Responsibilities pattern when your program needs to process different types of requests in various ways, but the exact types of requests and their sequence are not known in advance



(Shvets, Alexander, 2018)

Chain of Responsibility Design Pattern

- Structure:

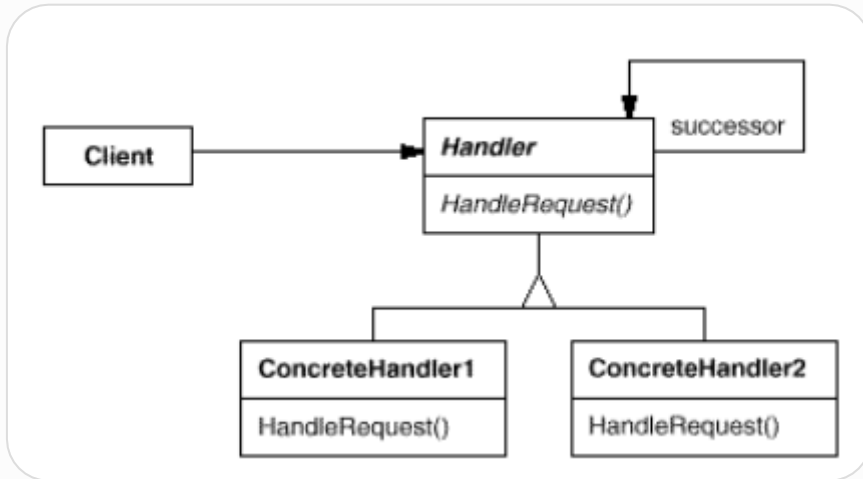
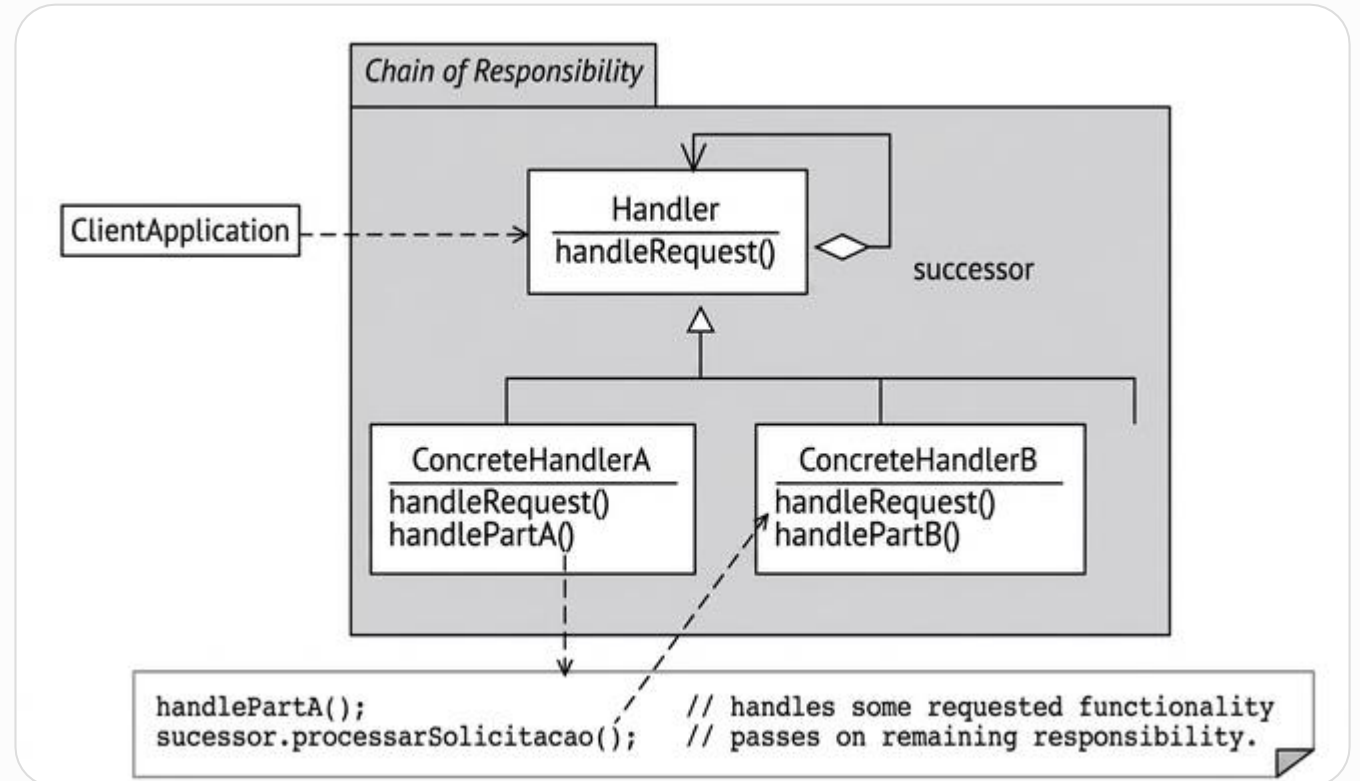


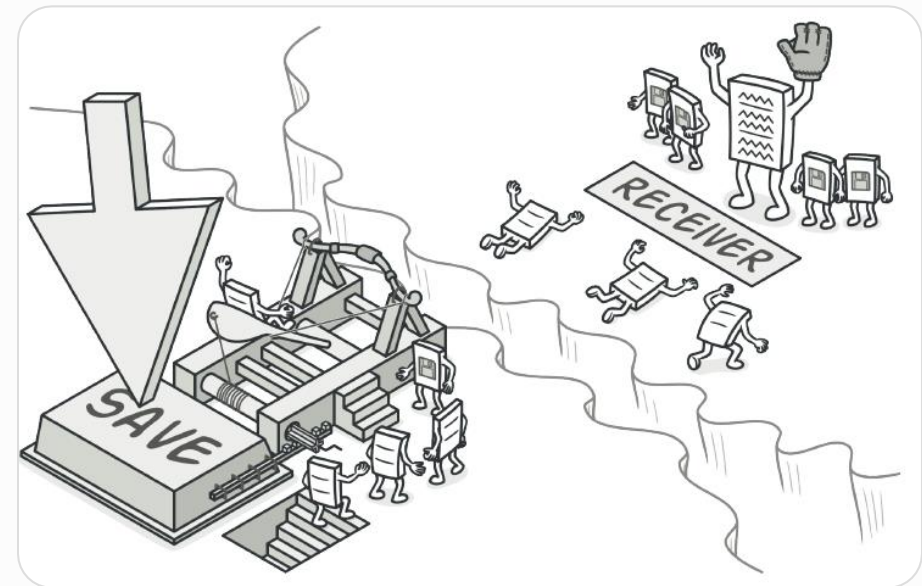
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Braude, 2005)

Command Design Pattern

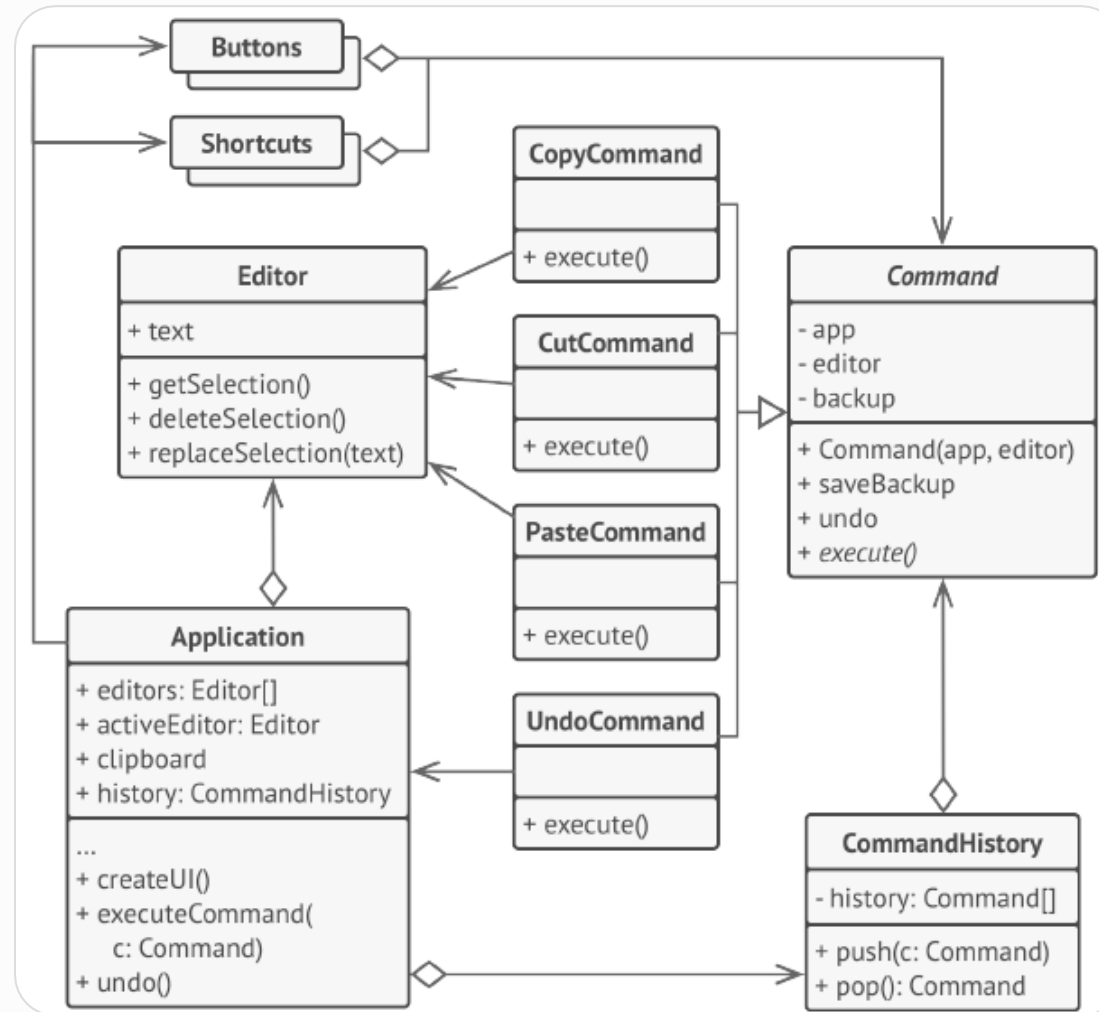
- **Classification:** Behavioral pattern/Object scope
- **Objective:** Encapsulate a request as an object, thereby allowing clients to be parameterized with different requests, create queues or logs of requests, and support undoable operations.
- **Motivation:** In some applications, you may need to parameterize objects with operations.



(Shvets, Alexander, 2018)

Command Design Pattern

- Example:



(Shvets, Alexander, 2018)

Command Design Pattern

- Structure:

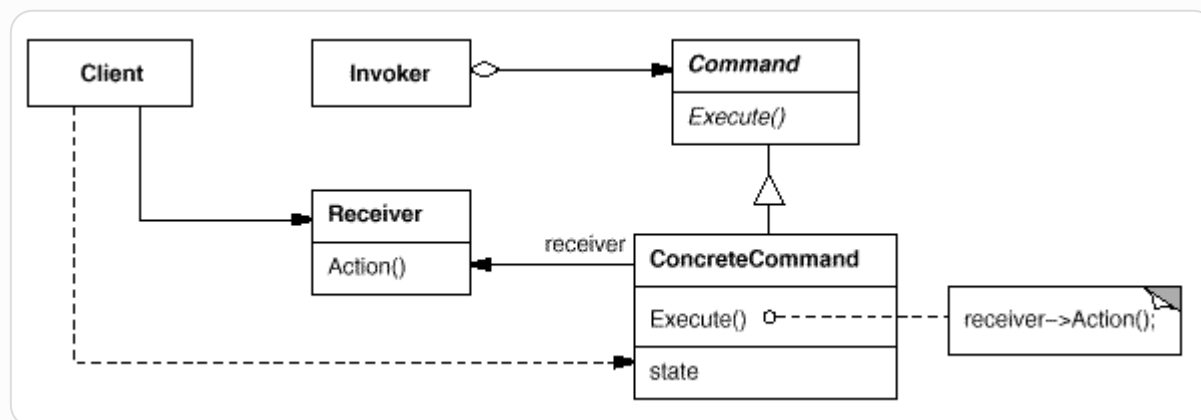
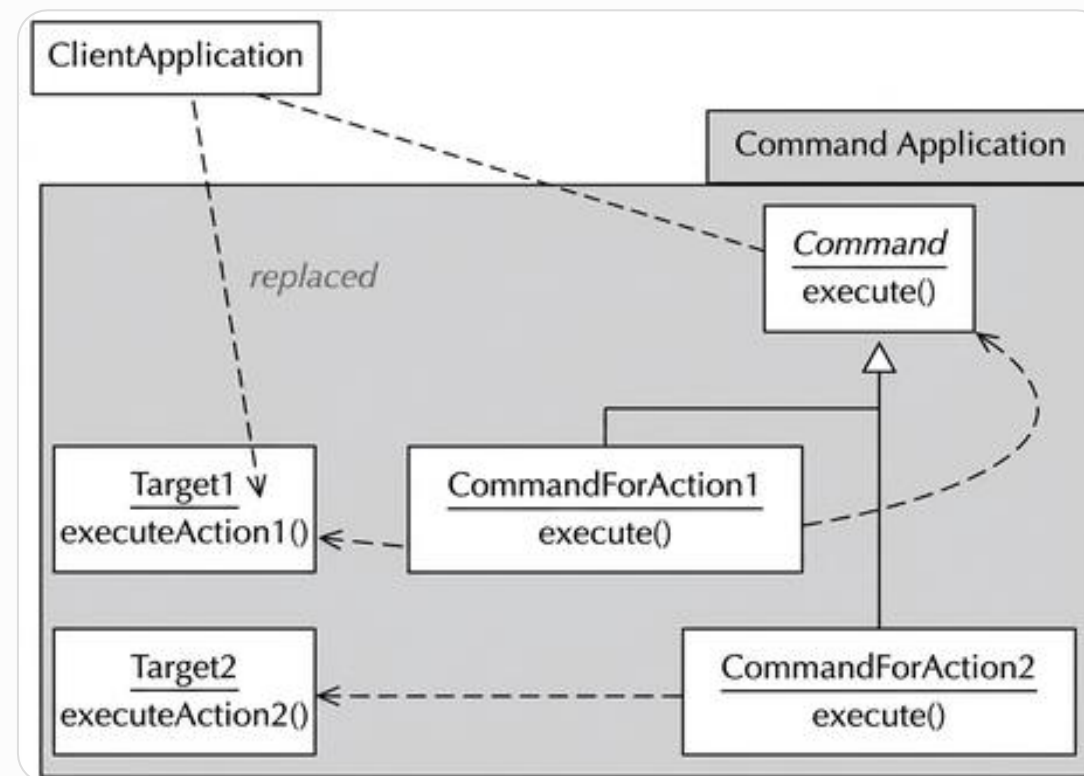


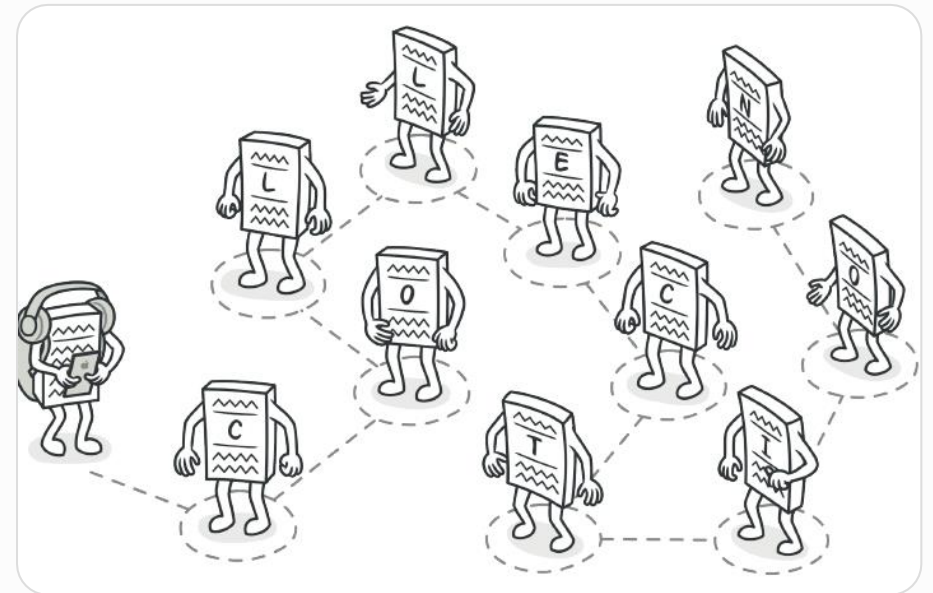
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Braude, 2005)

Iterator Design Pattern

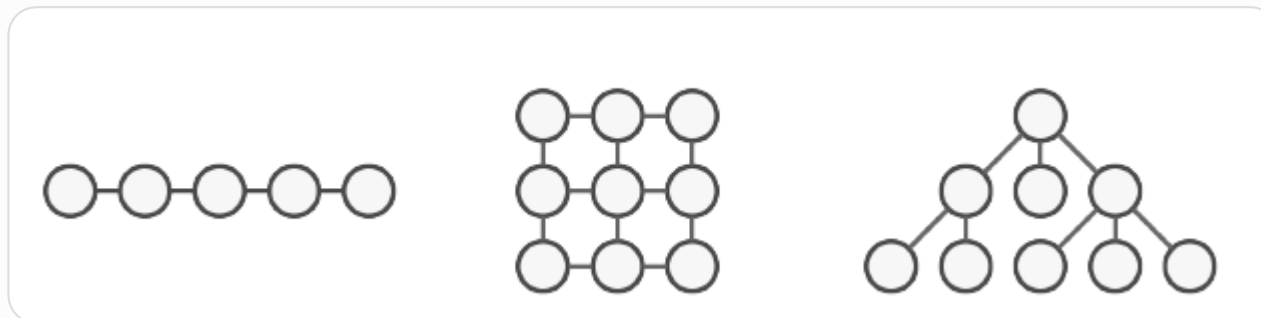
- **Classification:** Behavioral pattern/Object scope
- **Objective:** Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
- **Motivation:** There is a need to traverse any aggregates, in various orders, but each has a different way of being traversed. Including many methods to handle the iteration can pollute the interface. If it is desirable to traverse the aggregate in different ways, even more methods will be needed.



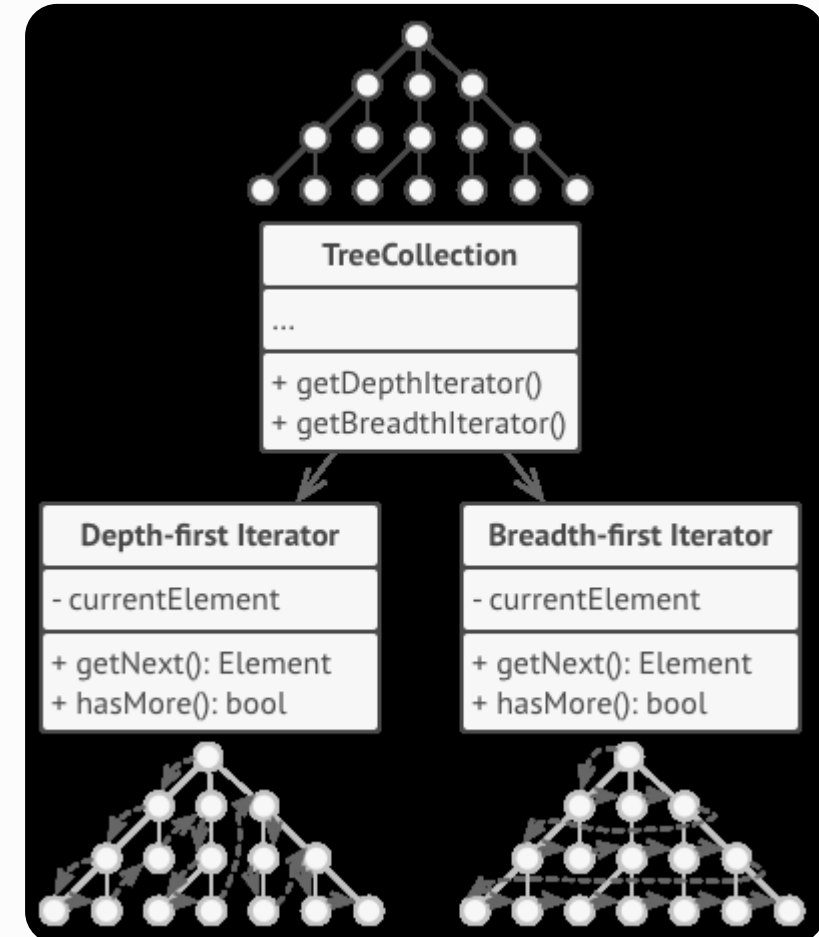
(Shvets, Alexander, 2018)

Iterator Design Pattern

- **Applicability:**
- Use the Iterator pattern to:
 - access the contents of an aggregate object without exposing its internal representation.
 - support multiple traversals of aggregate objects.
 - provide a uniform interface for traversing different aggregate structures (i.e., to support polymorphic iteration)



(Shvets, Alexander, 2018)



Iterator Design Pattern

- Structure:

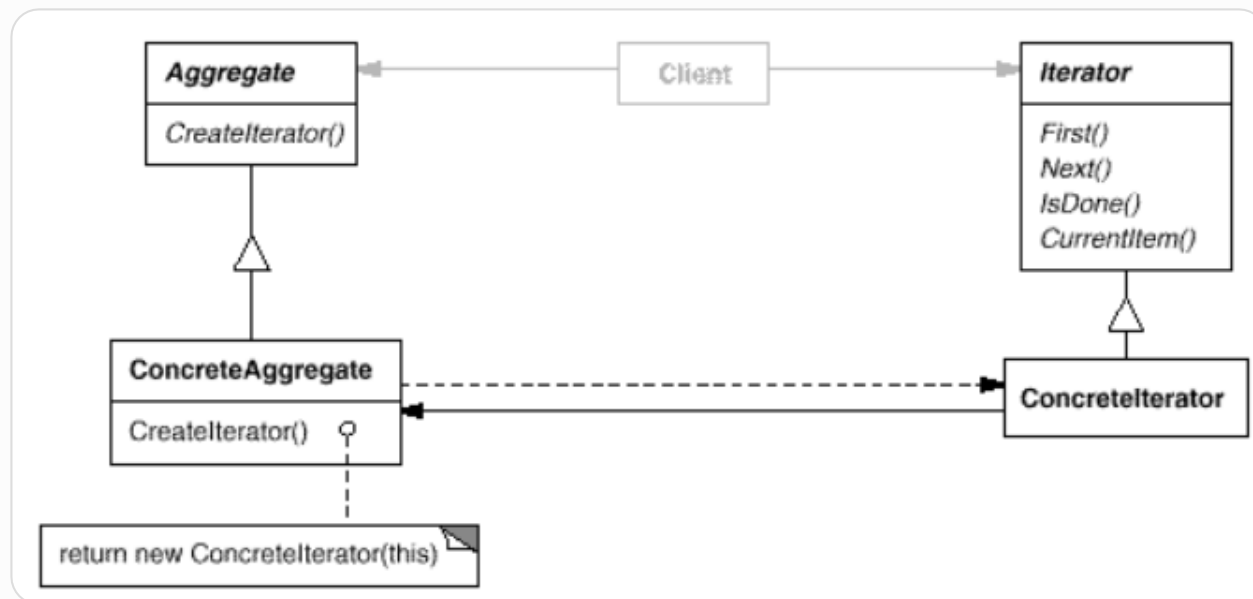
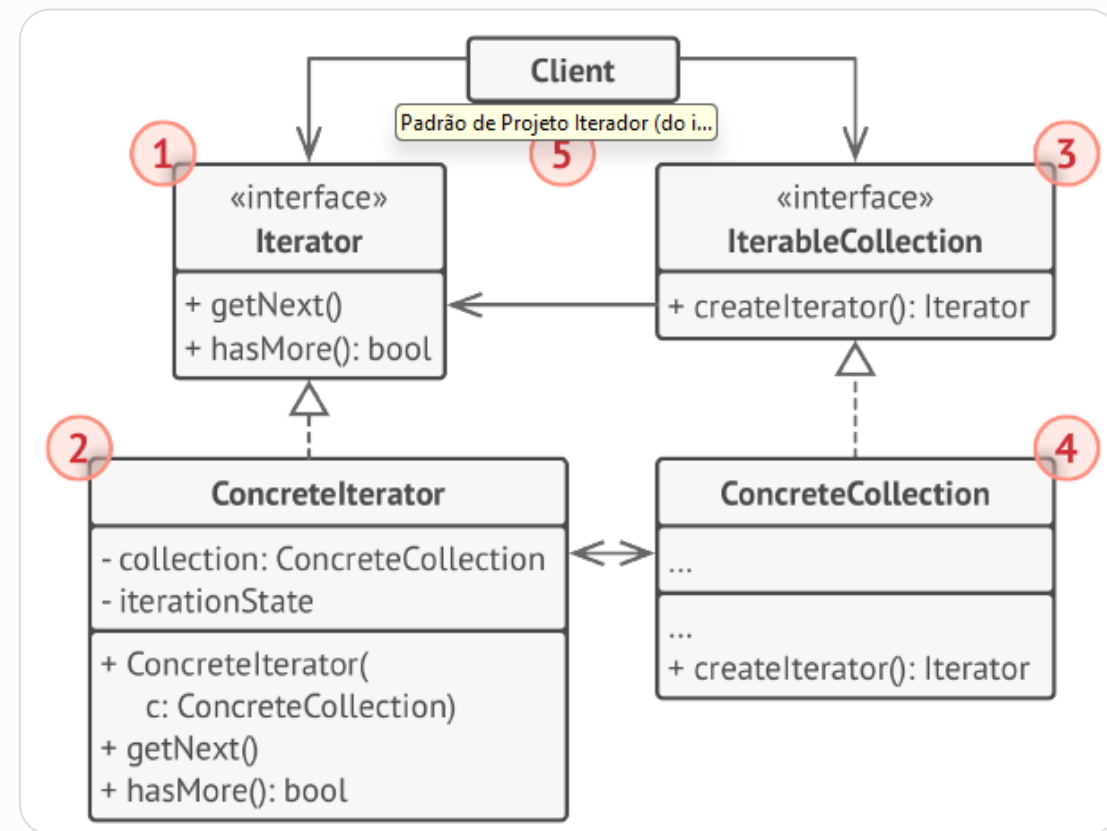


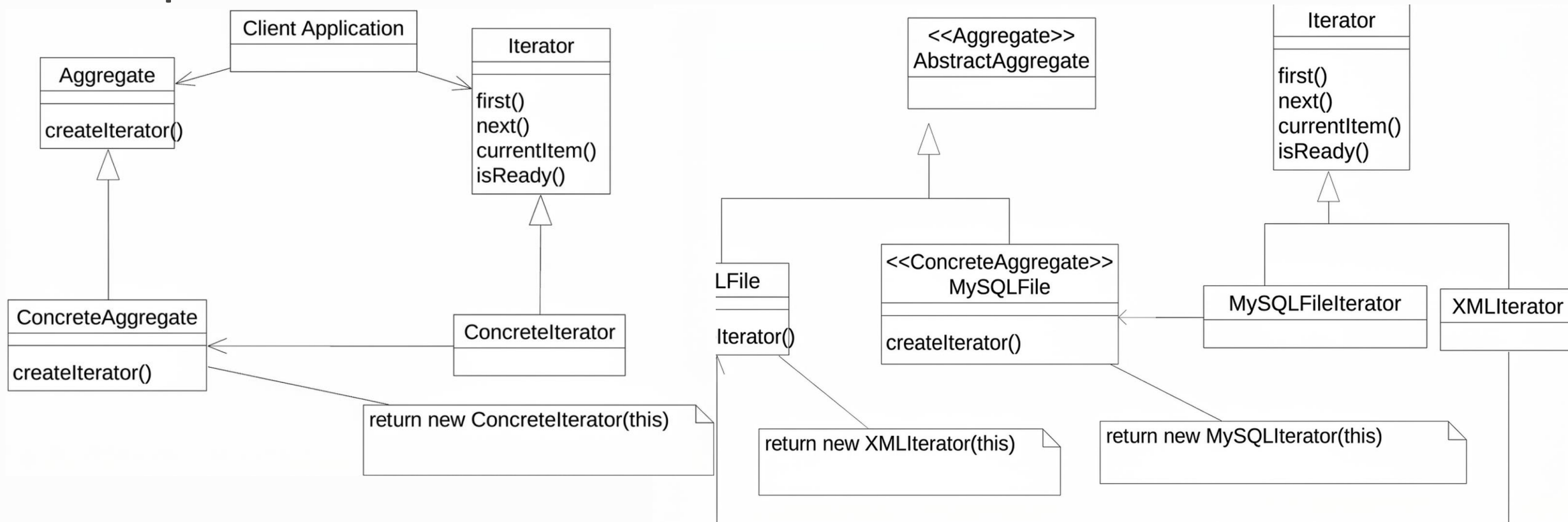
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Iterator Design Pattern

- Example:

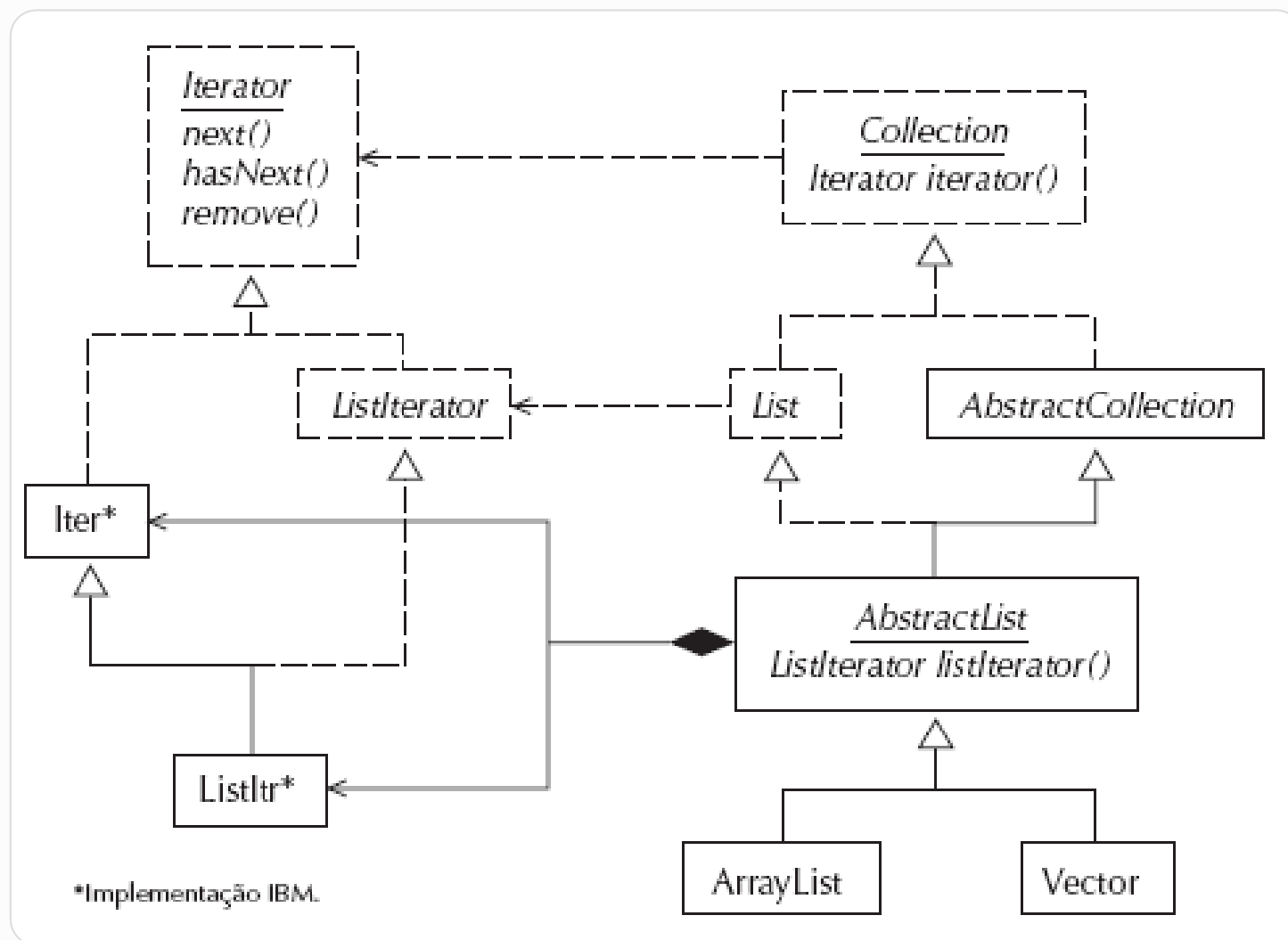


(Braude, 2005)

Iterator Design Pattern

- **Iterator in the Java API**
 - The java.util package contains an Iterator interface with the methods: next(), hasNext(), and remove().
 - The next() method is a combination of next() and currentItem()
 - The ListIterator subinterface specifies these methods to operate on List objects and adds appropriate methods to List objects.
 - The List interface has a listIterator() method, which returns a ListIterator object that iterates over it. This allows the programmer to move and iterate over List objects.

Iterator Design Pattern

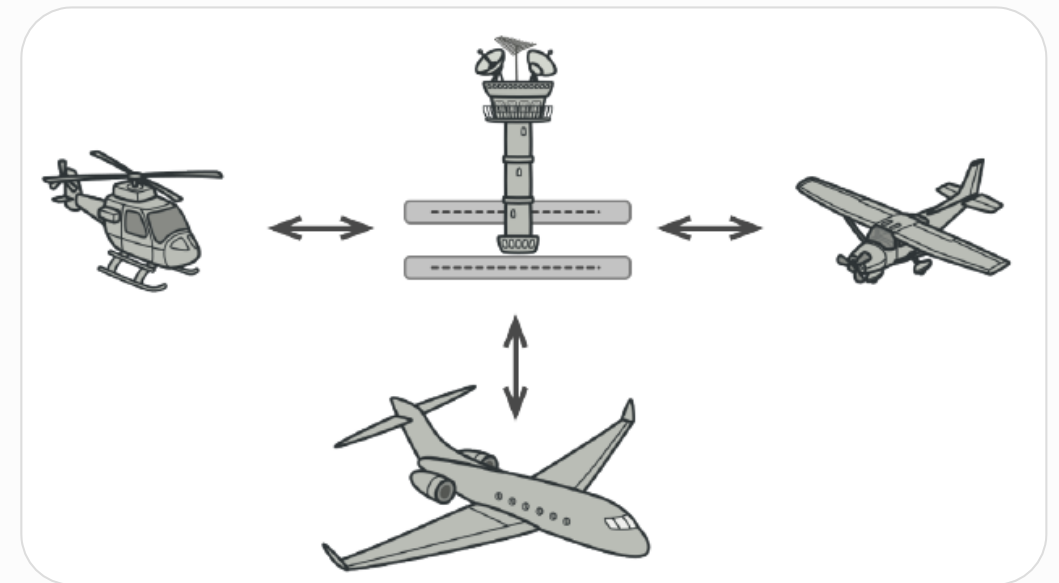


Iterator Design Pattern

```
public class Reader
{
    private String name;
    private Char[] type;
    private Boolean found=false;
    private List loans = new ArrayList();
    public void returnCopy(int copyCode)
    {
        Iterator i = loans.iterator();
        while (i.hasNext()) && (!found) {
            Loan e = (Loan) i.next();
            found=e.returnCopy(copyCode) }
        }
    }
```

Mediator Design Pattern

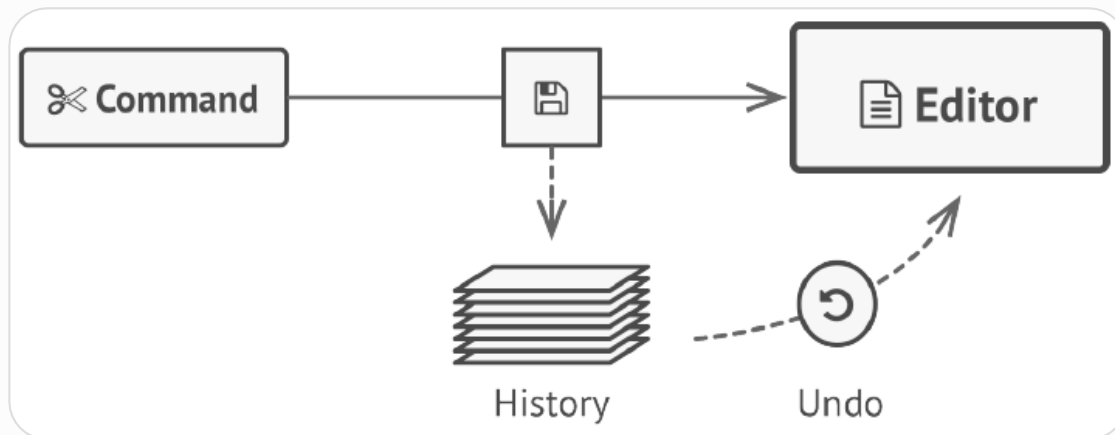
- **Classification:** Behavioral Pattern/Object Scope
- **Objective:** Define an object that encapsulates how a set of objects interacts. The Mediator promotes loose coupling by preventing objects from referring to each other explicitly, and allows you to vary their interaction independently.



(Shvets, Alexander, 2018)

Memento Design Pattern

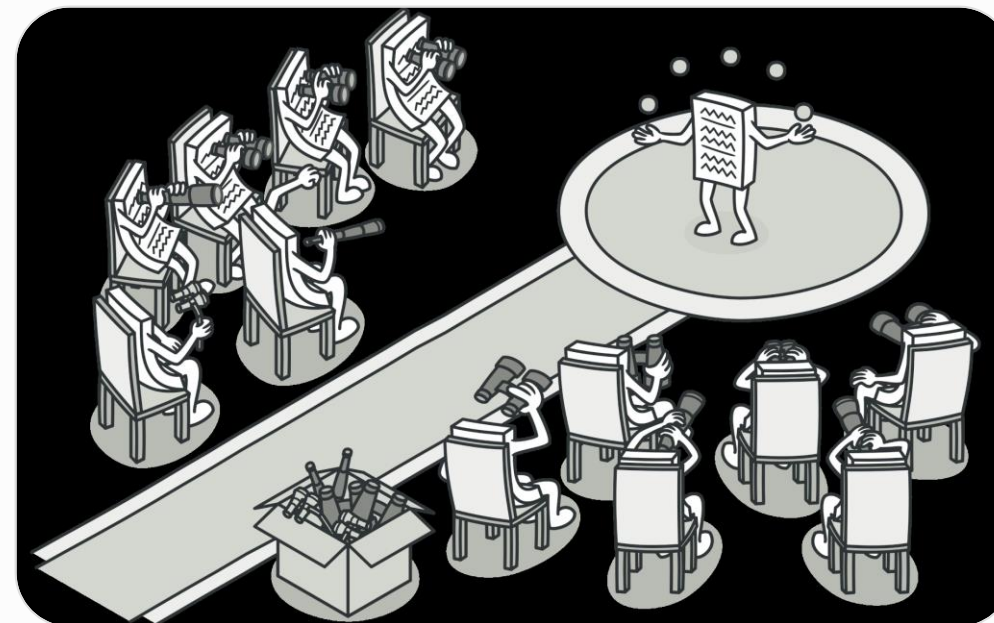
- **Classification:** Behavioral Pattern/Object Scope
- **Objective:** Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to that state later.



(Shvets, Alexander, 2018)

Observer Design Pattern

- **Classification:** Behavioral Pattern/Object Scope
- **Objective:** Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

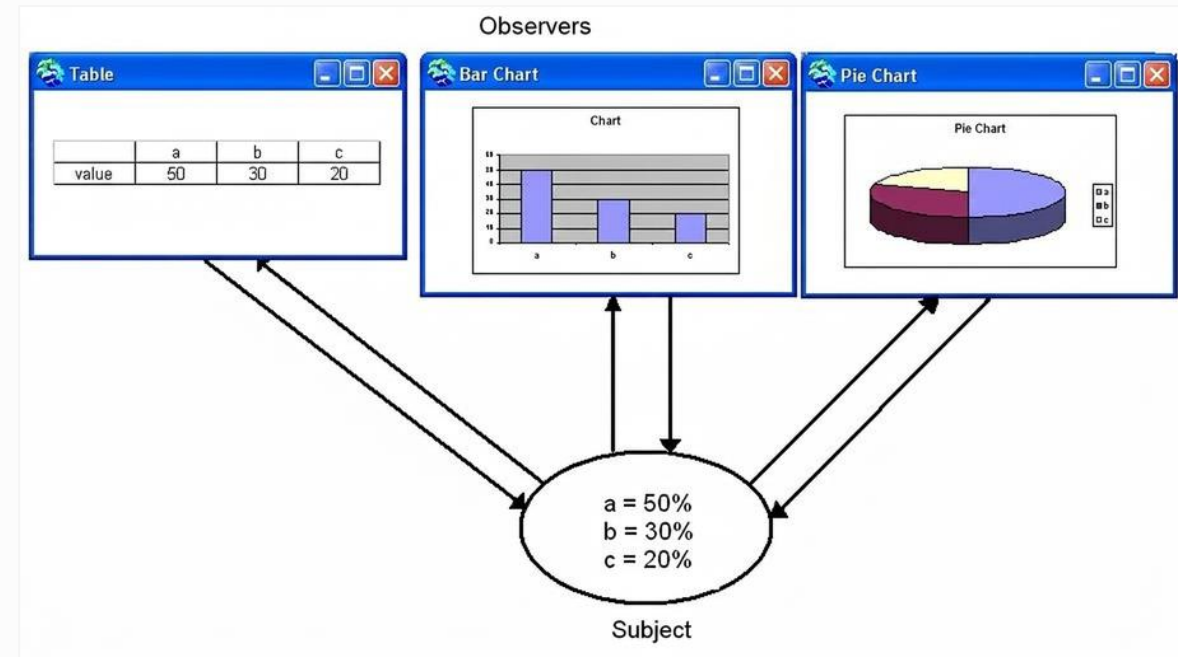


(Shvets, Alexander, 2018)

Observer Design Pattern

- **Applicability:**

- systems whose information is preferably centralized in a single source (subject), and there are several views (observers) that must be updated whenever the information is modified
 - Any attempt to maintain consistency increases coupling between classes.
 - For example, in an integrated tool, data itself can be separated from the various views of that data, so that a change in one view is reflected in the others, even if the views are unaware of each other.
- When an object must be able to notify other objects without making assumptions about who these objects are, or without knowing them beforehand, or when these observer objects change dynamically



Poll 9:

What does this remind you of?

Observer Design Pattern

- Structure:

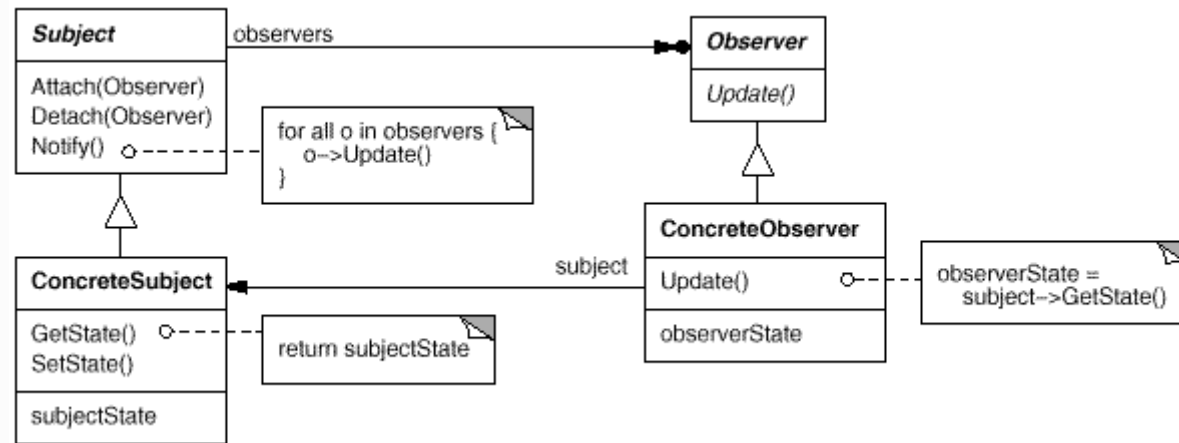
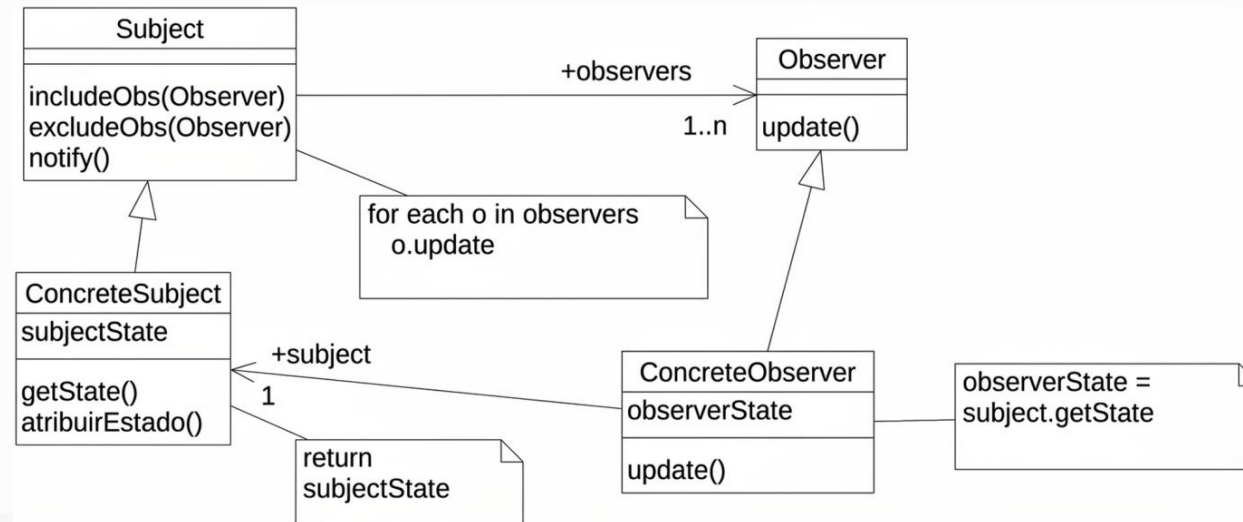


Image captured from the html version of the book published on CD (Gamma et al 1995)

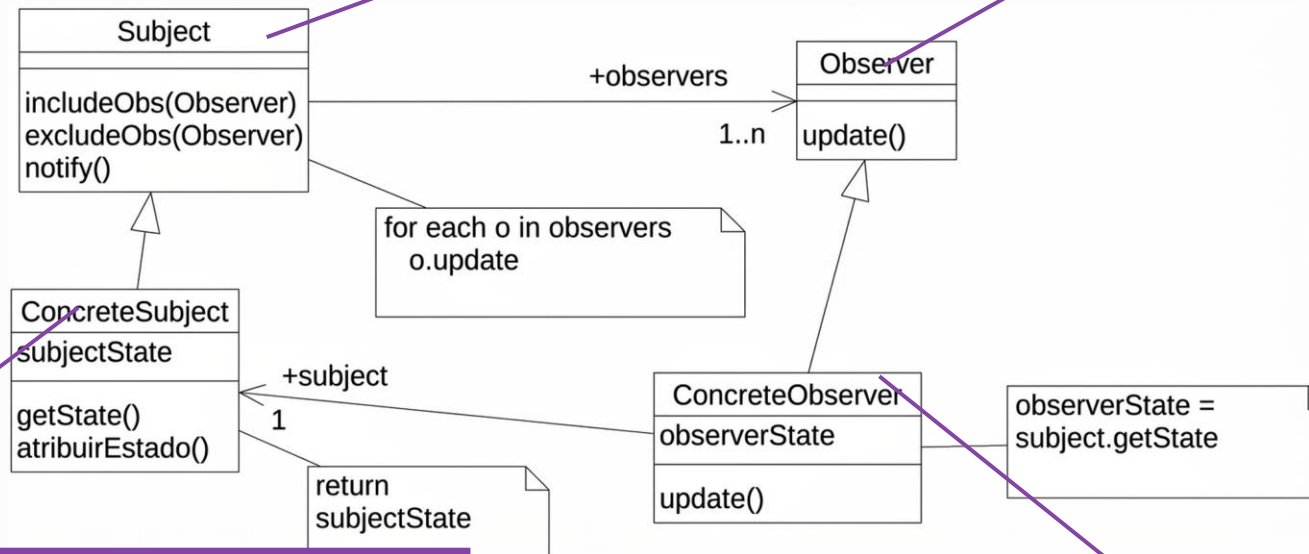


Observer Design Pattern

- **Participants:**

Knows its observers, provides an interface to include and exclude them

Defines an update interface for objects that will need to be notified of changes in the subject



- Stores the state of interest for ConcreteObserver objects.
- Sends a notification to its observers when its state changes

- Maintains reference to the ConcreteSubject object
- Stores the state that must be consistent with the subject
- Implements the abstract update method

Observer Design Pattern

- **Implementation methods:**
 - The observers themselves can send state updates to the concrete subject, which in turn notifies all observers
 - The other observers can get the state to know if they need to update or not
 - Another form of implementation would be for observers, upon subscribing, to specify some conditions so that they only receive the notification if the conditions are true
 - In Gamma's book there are several other important considerations to take into account when implementing this pattern

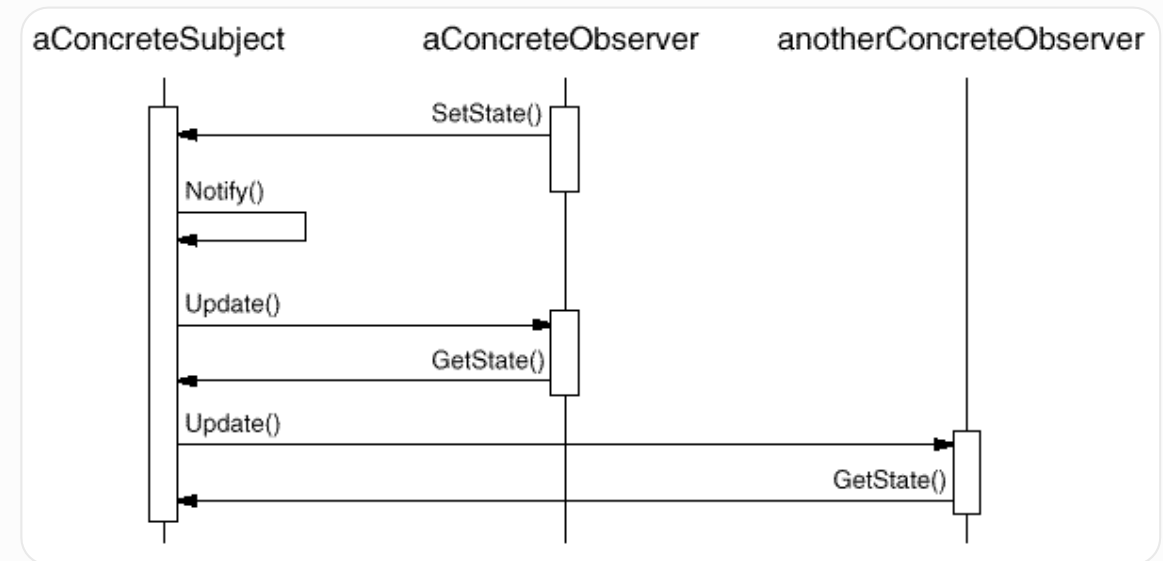
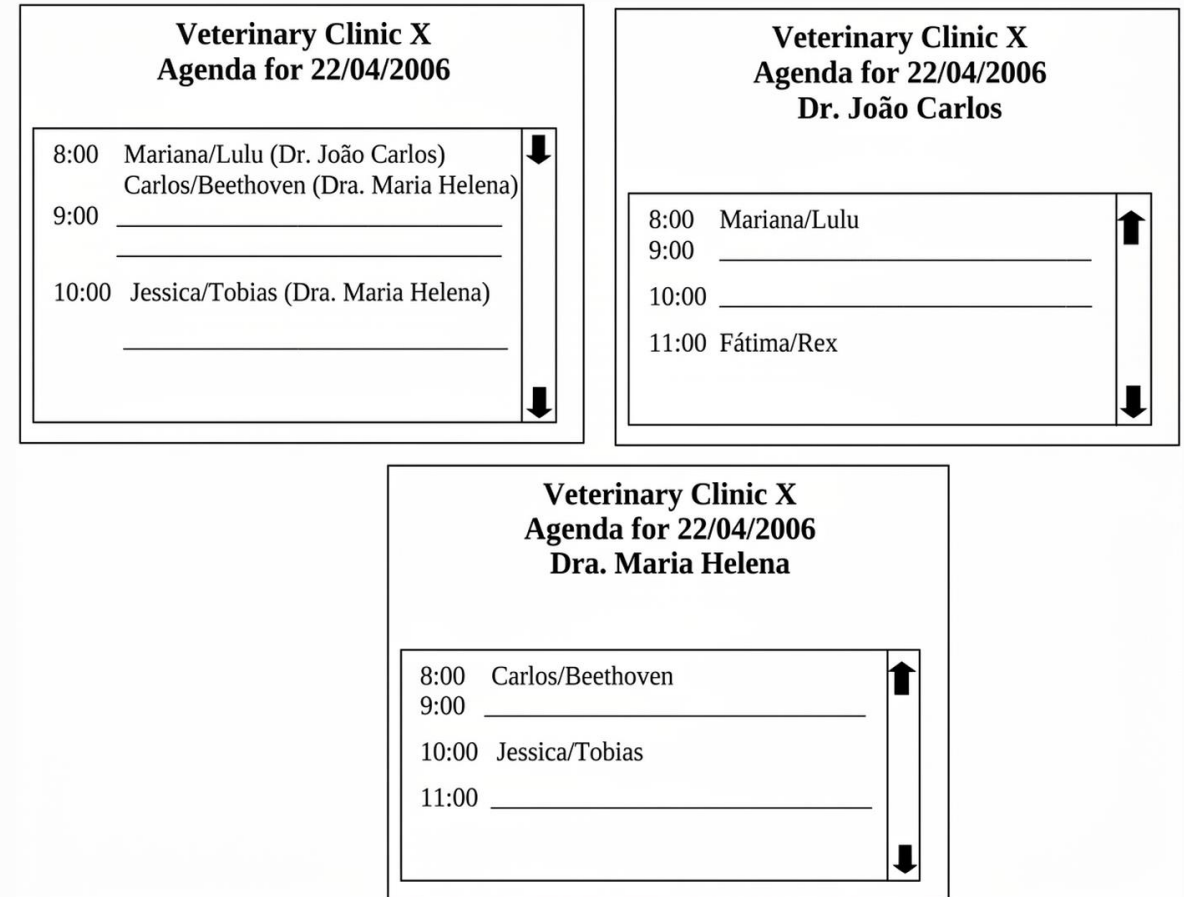


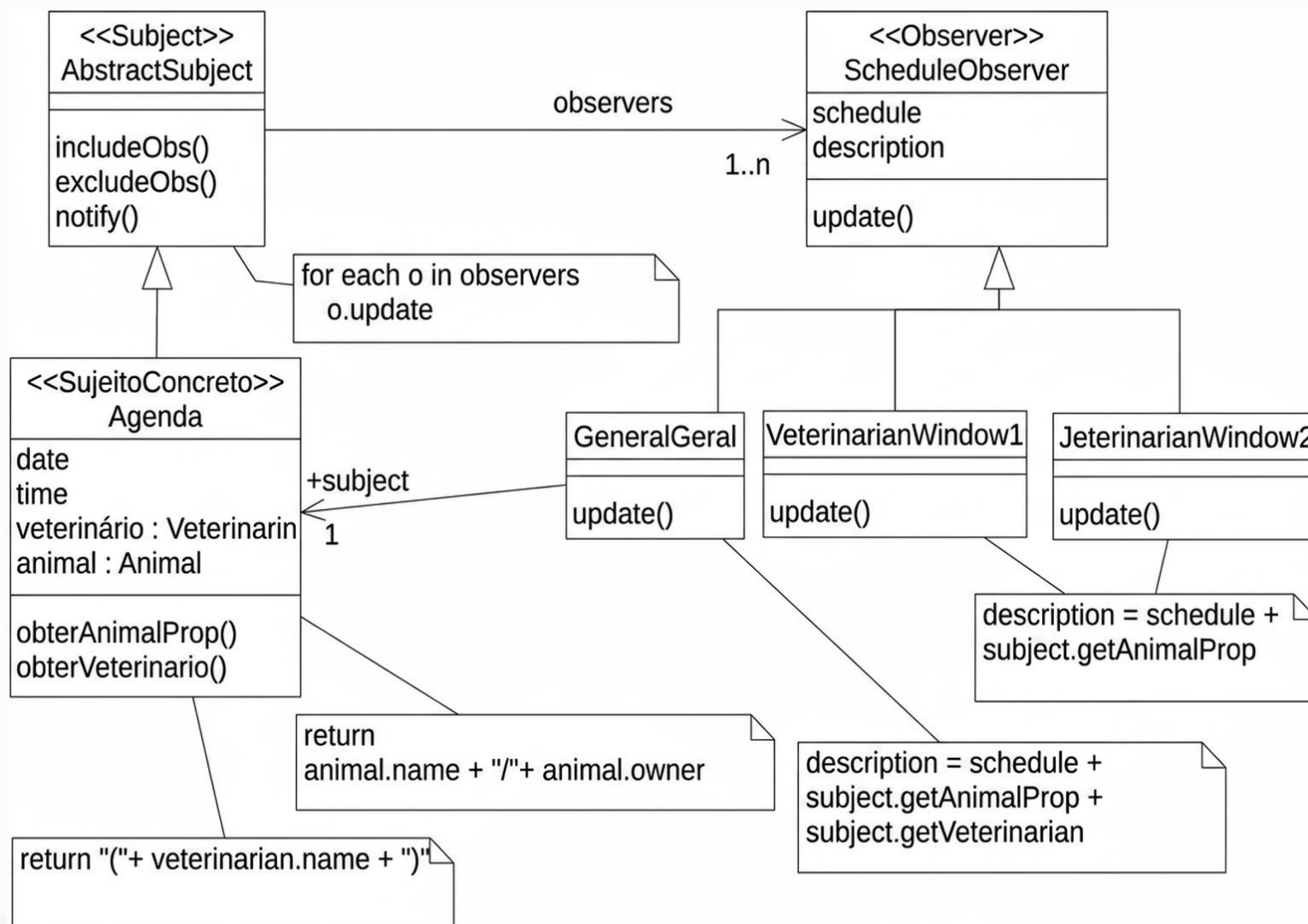
Image captured from the html version of the book published on CD (Gamma et al 1995)

Observer Design Pattern

- **Example:**
- design the graphical user interface for a scheduling system assigned to the various veterinarians working in a clinic.
- In the secretary's view, all appointments should appear
- In each veterinarian's view, only their private schedule appears

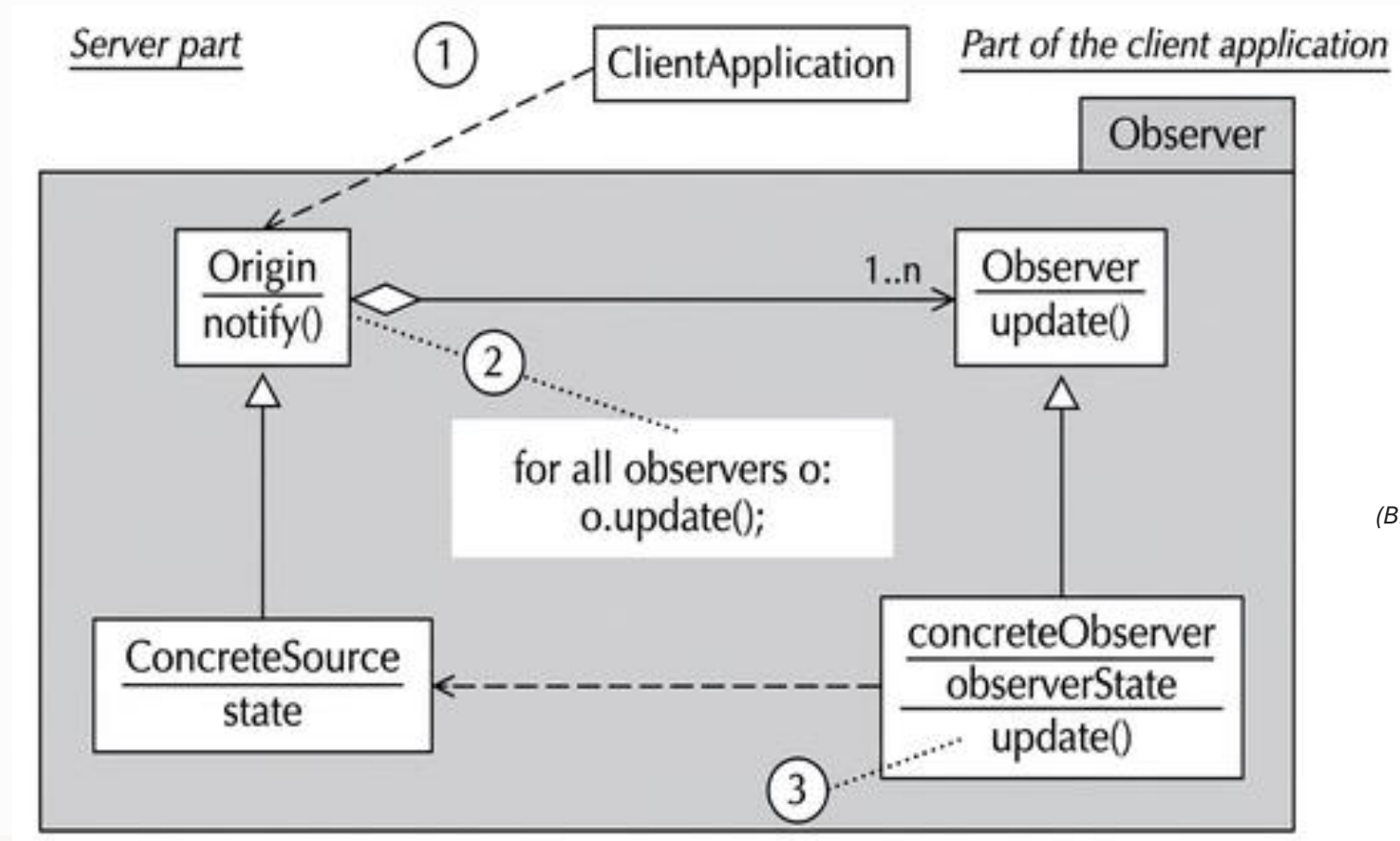


Observer Design Pattern



Observer Design Pattern

- How does the Observer work?

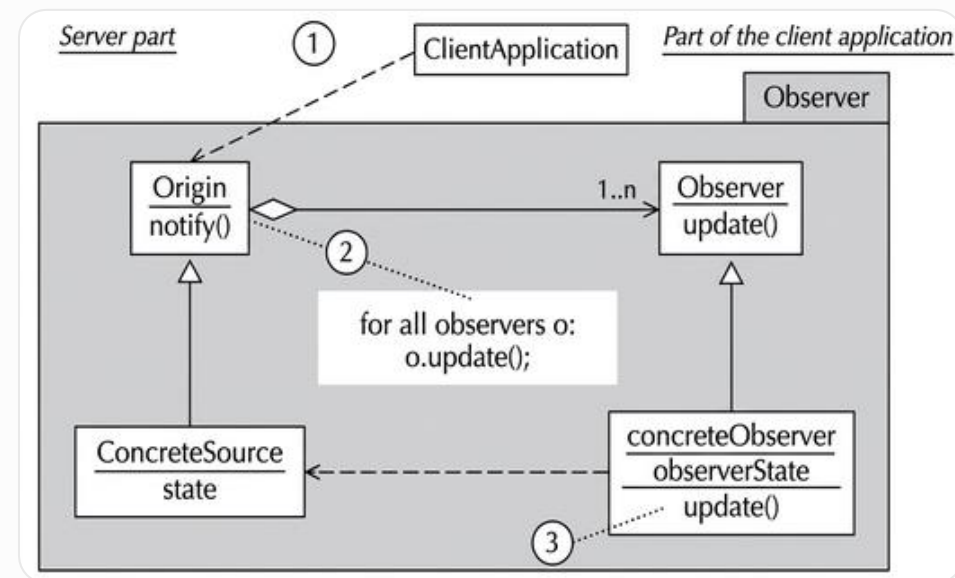


(Braude, 2005)

Observer Design Pattern

- **How does it work?**

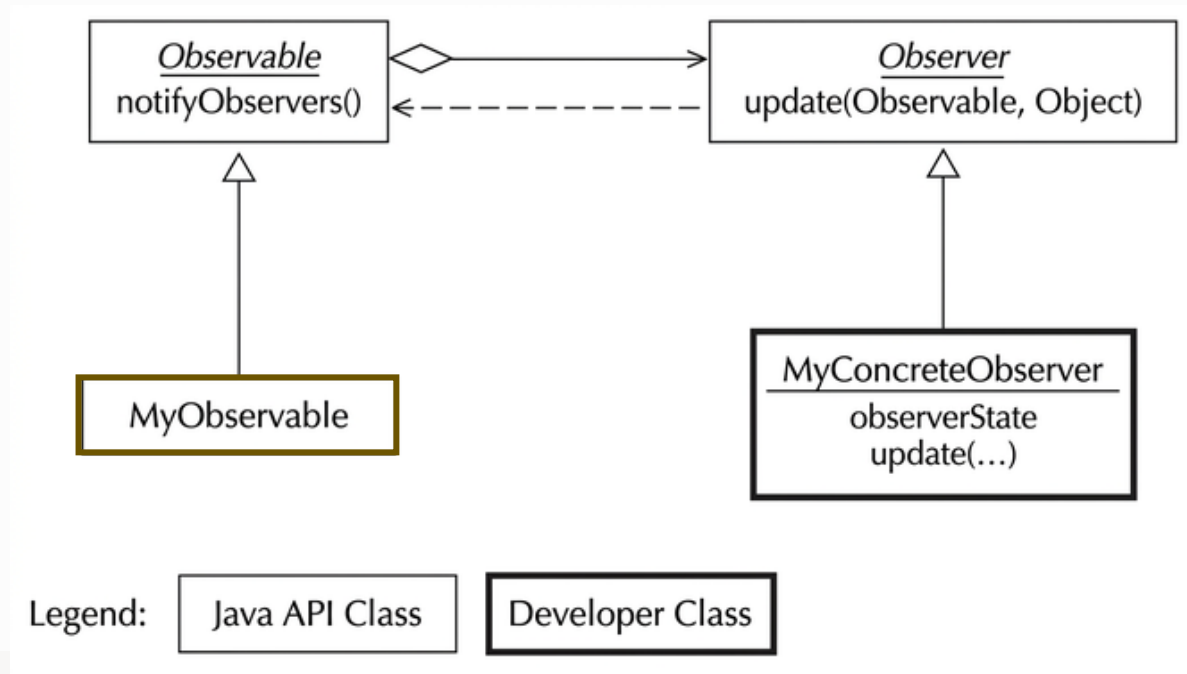
- (Step 1) The client application references a known interface object, requesting that observers be notified.
 - For example, the client application could be a process scheduled to alert about a data change. In the model, this is shown as a ClientApplication object, which informs the Source object to execute its notify() function.
- (Step 2) The notify() method calls the update() function on each Observer object it aggregates.
- (Step 3) The implementation of update() depends on the particular ConcreteObserver it belongs to.
 - Normally, update() compares the state of the ConcreteObserver object (variable values) with that of the central data source, to then decide whether or not to change its variable values in the same way.



(Braude, 2005)

Observer Design Pattern

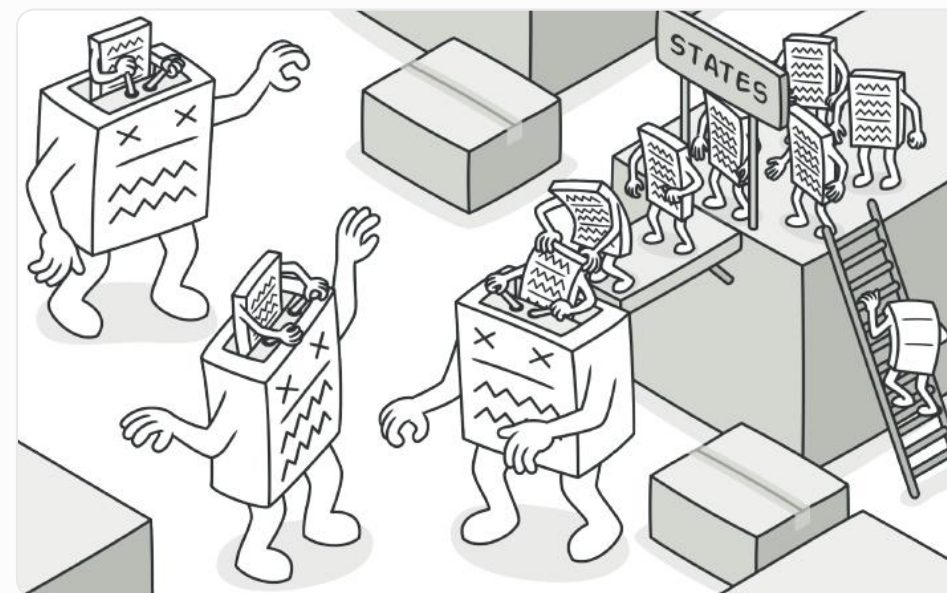
- It is worth remembering that this pattern exists in the Java API
 - Note that `update()` is a call-back method, because it provides Observer objects with a reference to their source, allowing them to compare their data, for example with the Observable object when executing the update.
 - Since the update is done retroactively, there is no need for concrete Observer classes to maintain references to the Observable object.



(Braude, 2005)

State Design Pattern

- **Classification:** Behavioral Pattern/Object Scope
- **Objective:** Allow an object to change its behavior when its internal state changes. The idea is to make it seem as if the object has changed class.
- **Applicability:**
 - when an object's behavior depends on its state, which can change at runtime
 - when this behavior change implemented using case structures favors errors, especially when the system evolves and new states emerge, as several operations will contain these case structures.



(Shvets, Alexander, 2018)

State Design Pattern

- **Structure:**

- Create one class to represent the object itself and another to represent its state
- The state can vary independently of other objects.

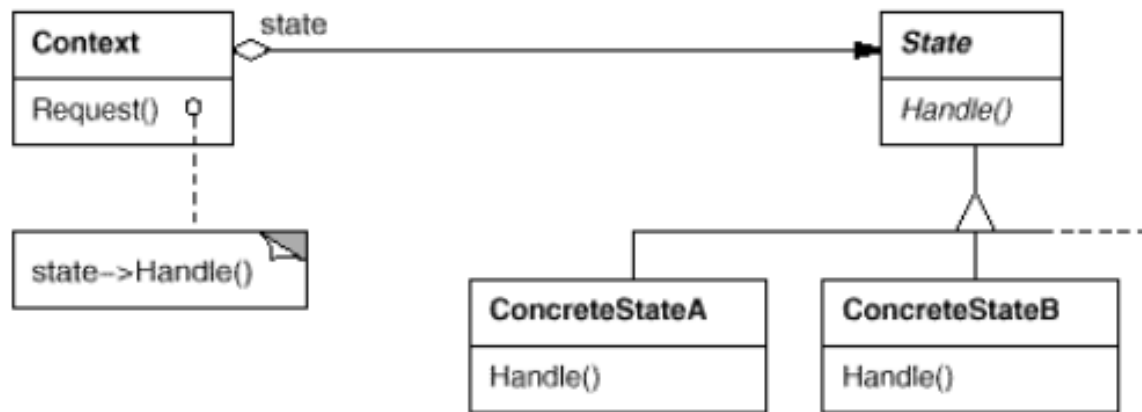
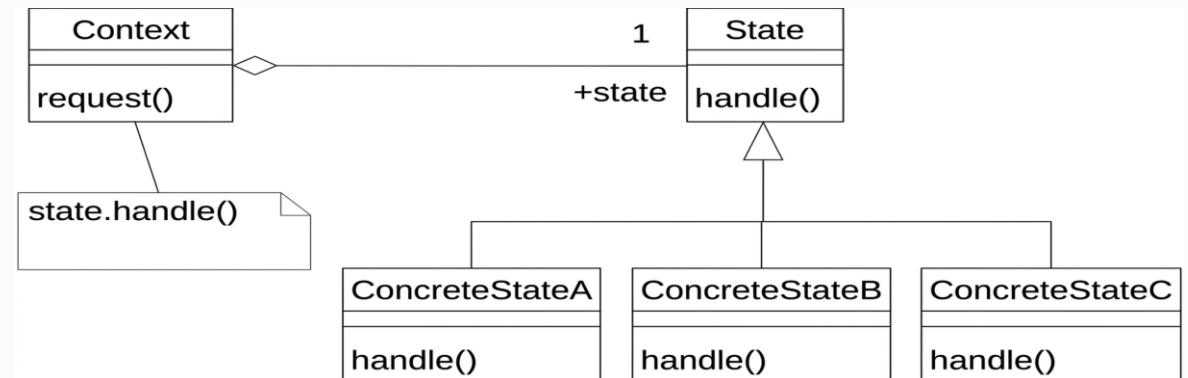


Image captured from the html version of the book published on CD (Gamma et al 1995)

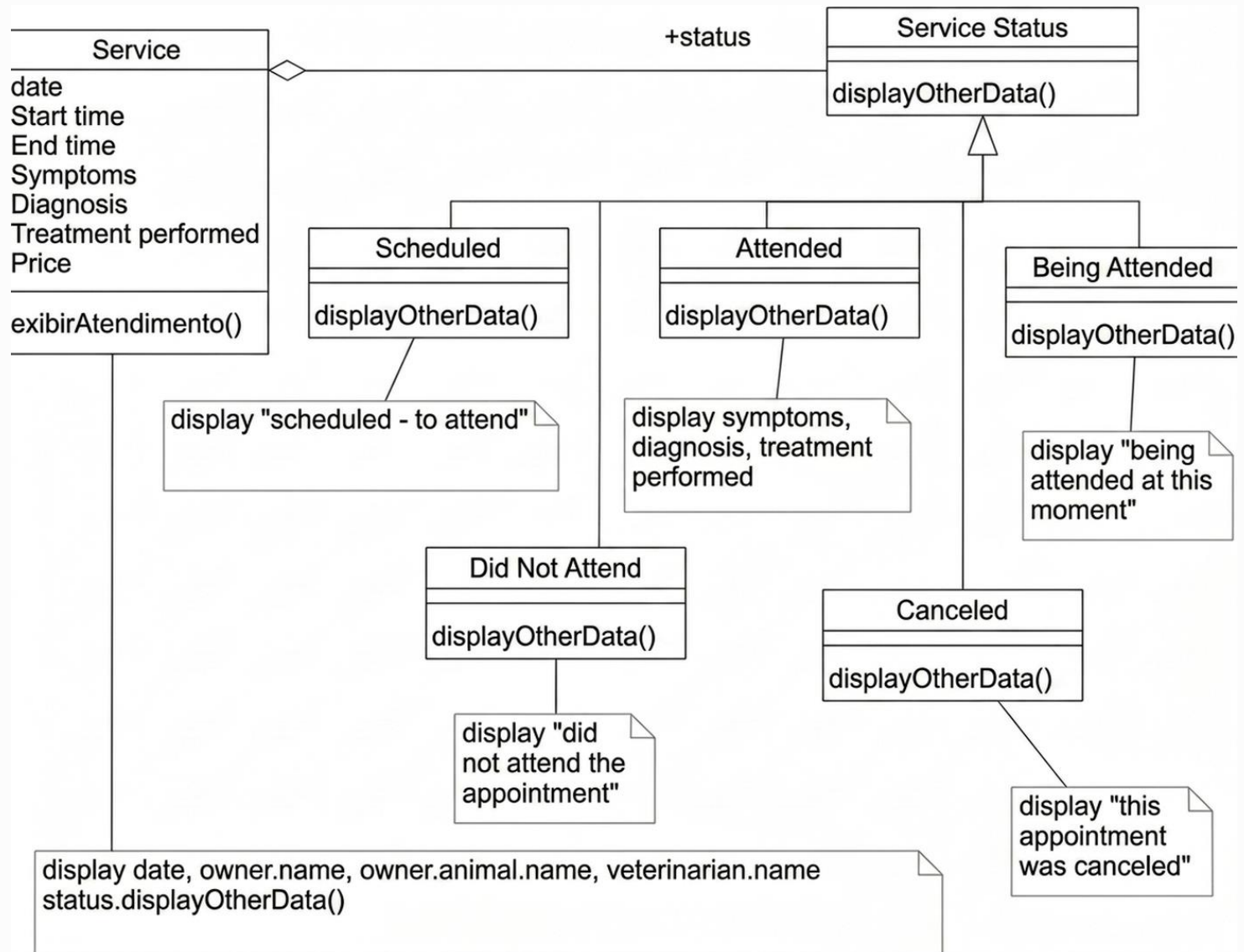


State Design Pattern

- **Example:**
 - Display a message that has a fixed part and another part that depends on the patient's service status

Poll 10:

Use of the State pattern



State Design Pattern

- **Implementation issues:**
 - Who defines state transitions: context or state?
 - If there is a fixed criterion for state change: context
 - Constant changes: state
 - Interface needed so that state objects can explicitly modify the current context state
 - Table to map transitions
 - Create and destroy State objects
 - Create all objects a priori or create/destroy on demand?

State Design Pattern

- Example of State usage in a game:
 - [State · Design Patterns Revisited · Game Programming Patterns](#)

Strategy Design Pattern

- **Classification:** Behavioral pattern/Object scope
- **Objective:** Define a family of algorithms, encapsulate each algorithm, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Applicability:**
 - several related classes differ only in their behavior, or
 - multiple versions of an algorithm are needed, or
 - client applications do not need to know specific details of each algorithm's data structures



(Shvets, Alexander, 2018)

Strategy Design Pattern

- **Example:**

- Create an abstract class for the Strategy employed by the algorithm, as well as subclasses specializing each of the algorithms.
- The Context maintains a reference to the Strategy object and can define an interface to allow the Strategy to access its data. The Strategy defines a common interface for all available algorithms. The Context delegates requests received from client applications to its strategy.

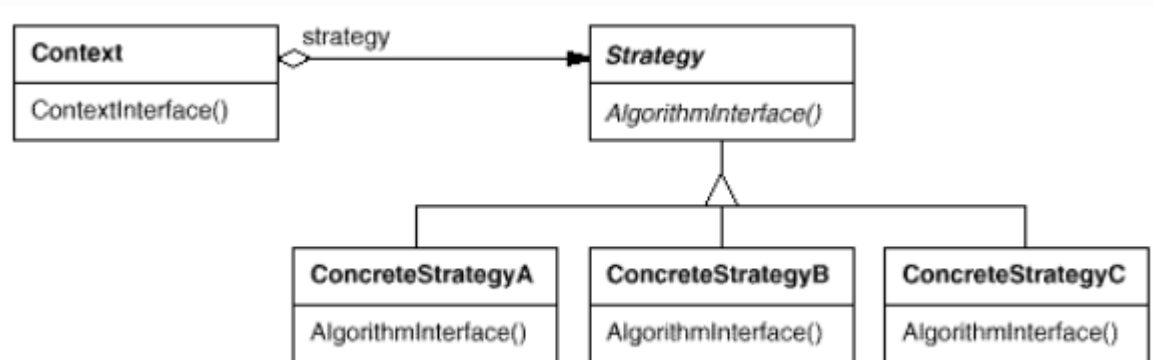
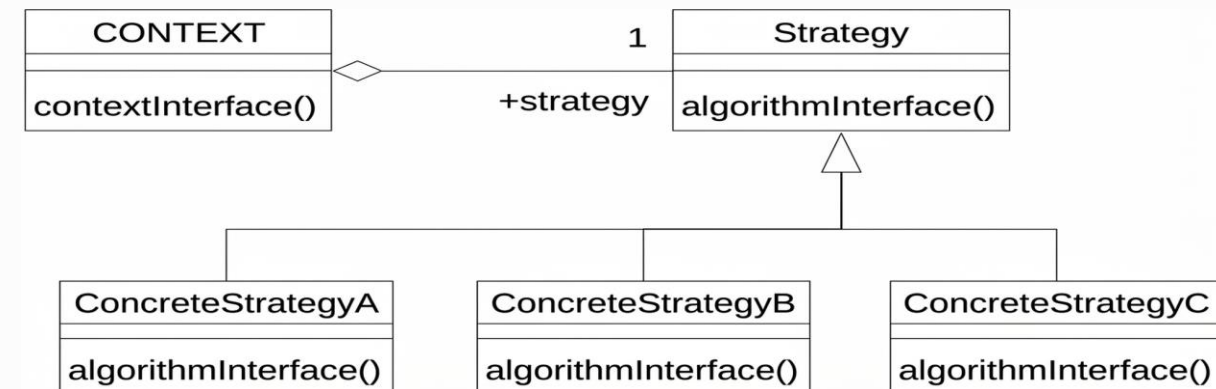
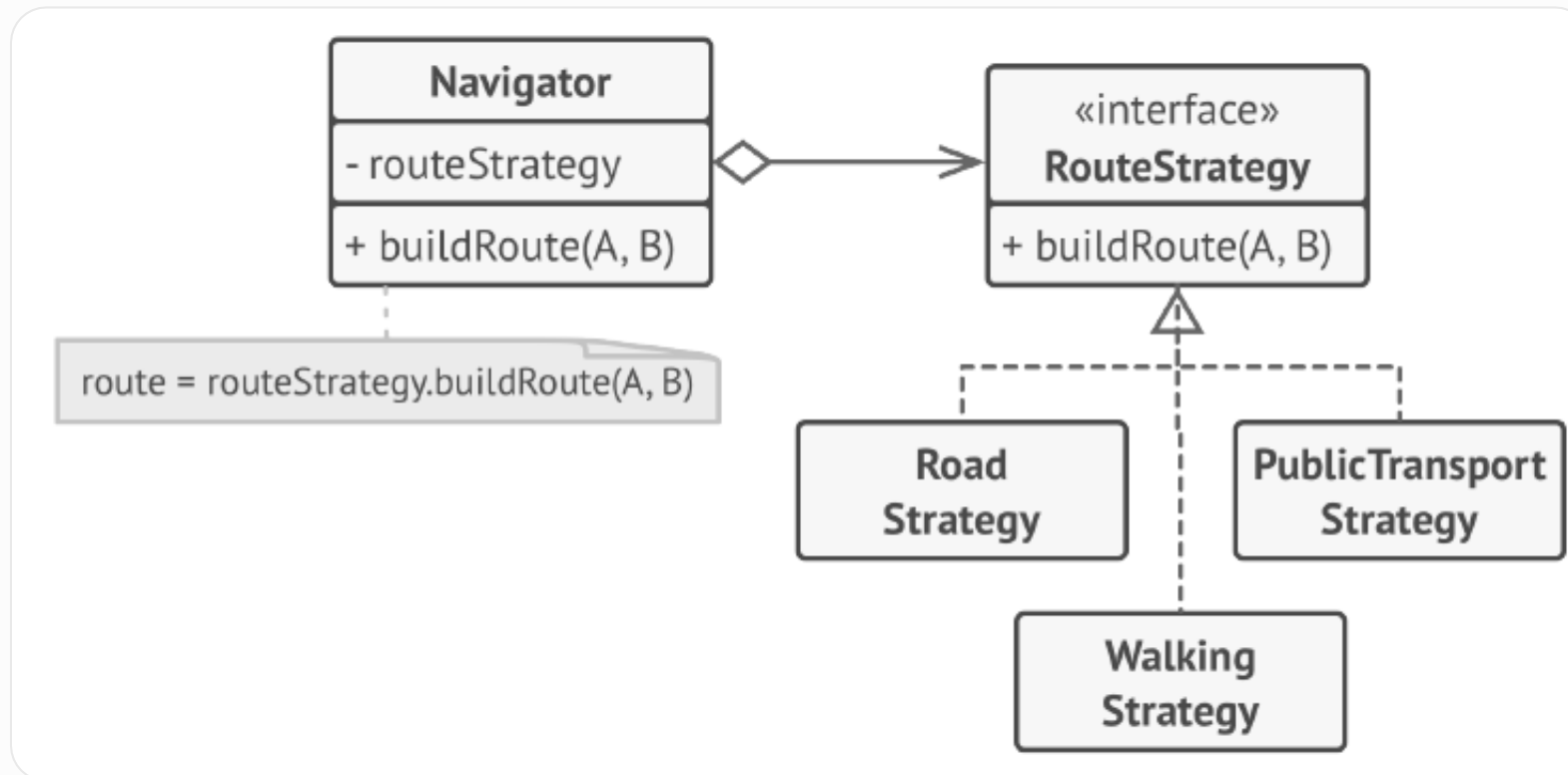


Image captured from the htm version of the book published on CD (Gamma et al 1995)



Strategy Design Pattern

- **Example:** Route calculation algorithm depends on what the user chooses



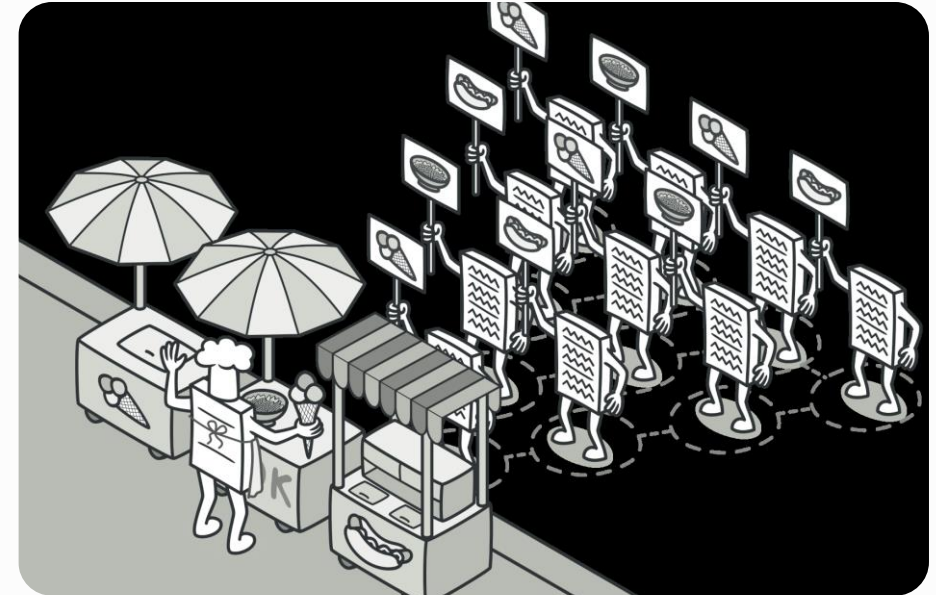
(Shvets, Alexander, 2018)

Difference between State and Strategy?

- State can be considered an extension of Strategy.
- Both patterns are based on composition: they change the context's behavior by delegating some work to helper objects.
- Strategy makes objects completely independent and unaware of each other.
- However, State does not restrict dependencies between concrete states, allowing them to change the context's state at will.

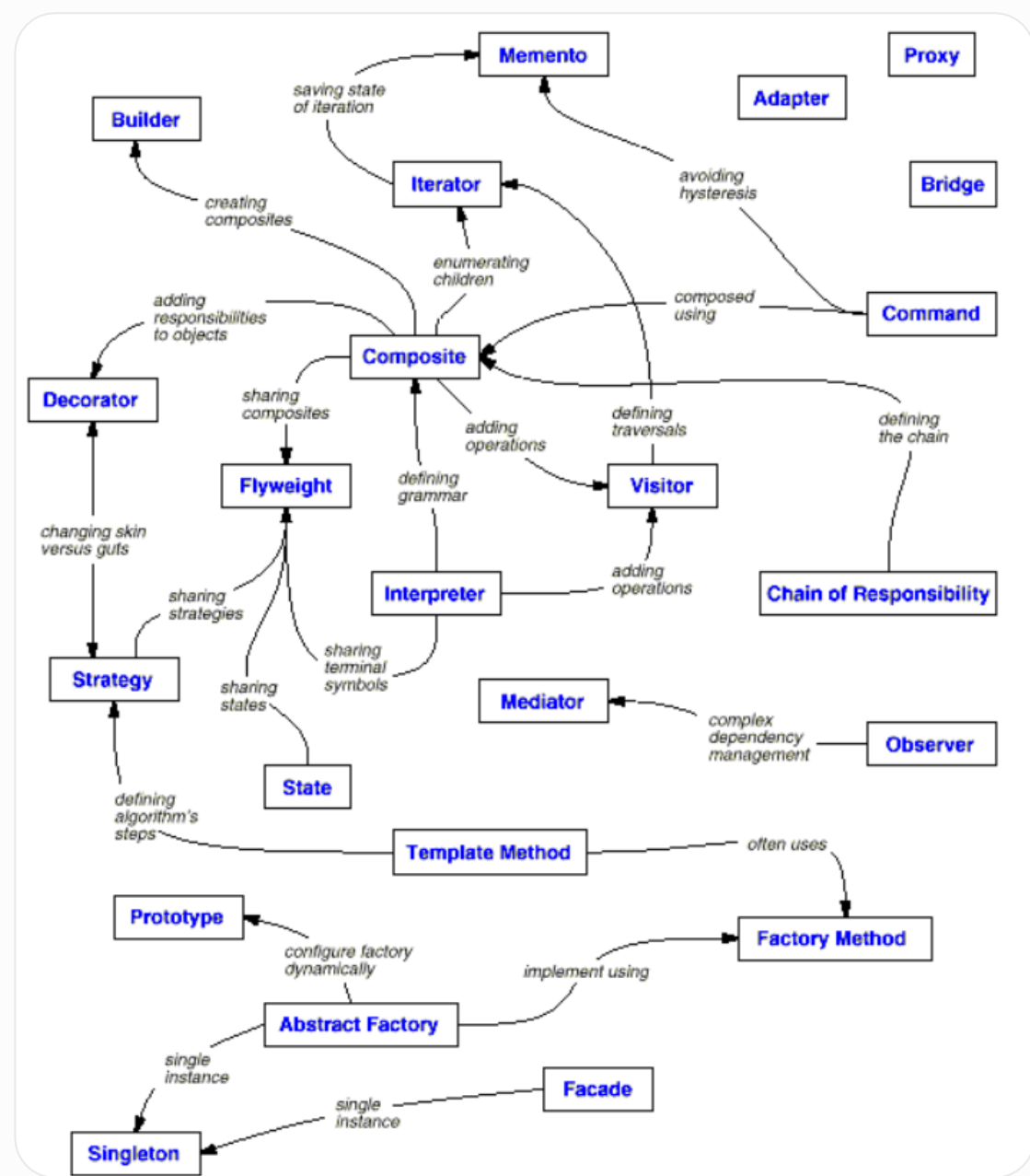
Visitor Design Pattern

- **Classification:** Behavioral pattern/Object scope
- **Objective:** Allows an operation to be performed across a set of objects of different classes, with a “visitor” object implementing variants of the same operation, corresponding to all target classes.
 - Also use this pattern when a behavior makes sense only in some classes of the hierarchy, but not in others



(Shvets, Alexander, 2018)

Relationships between Design Patterns



Conclusions

Advantages:

- Solutions used in various projects and proven useful
- Solutions with known advantages and disadvantages
- Offer a common vocabulary among developers
- Adhere to object-oriented best practices
- Suggest extensive, flexible, and robust systems

Disadvantages:

- Creation of various auxiliary classes and interfaces, leading to more complexity
- May generate memory and processing overhead
- May be worse than simple solutions when used in wrong contexts

Discussion: what is the impact of Artificial Intelligence on software development using design patterns?

- Some visible impacts:
 - New AI-specific design patterns: multi-agents, prompt chaining;
 - Adaptation of traditional design patterns: For example:
 - 1) heavy AI models may require adaptations in patterns like Factory and Builder;
 - 2) the need for agility of AI agents to respond dynamically to changing conditions may require changes in patterns like Observer and Strategy.
- Impact of patterns on sustainability (increased energy consumption)

Discussion: what is the impact of Artificial Intelligence on software development using design patterns?

- Changes in the development cycle
 - Assistance in adopting design patterns: AI tools can automatically incorporate these patterns into the code, improving architectural choices;
 - new possibilities to effectively automate the process of recognizing, suggesting, and adapting software design patterns, resulting in overall development improvement;
 - Maintenance problems: AI-generated code can be more difficult to understand/maintain
 - serious problems can occur if the patterns are not understood by the developer responsible for maintenance (increase in technical debt).

Discussion: what is the impact of Artificial Intelligence on software development using design patterns?

- Role of the software engineer
 - Ceases to be a “coder” to become an architect and orchestrator (importance of good software specification);
 - Human judgment is more critical to ensure that AI-generated code adheres to fundamental design principles, including design patterns;
 - It is essential that the software engineer ensures the code remains maintainable (reducing technical debt).

Discussion: what is the impact of Artificial Intelligence on software development using design patterns?

- In conclusion: AI can help leverage the use of patterns, as they are based on known uses to solve real problems;
- Since the emergence of software patterns, it has been said that, in the future, no one would need to remember them, as they would have been naturally incorporated into development.
- AI can help with this, but the role of the software engineer in judging whether the pattern was well applied remains important (at least for now).

References

- Biggerstaff, Ted J. ; Perlis, Alan J. **Software Reusability: Concepts and Models**. Acm Press Frontier Series, 1989
- Alexander et al. **A Pattern Language**. New York: Oxford University Press, 1977.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. **Design Patterns - Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1995
- Meszaros, G.; Doble, J. **A pattern language for pattern writing**, cap. 29 in J. Coplien; D. Schmidt. Pattern Languages of Program Design, Reading-MA, Addison-Wesley, p. 529–574, 1998.
- Buschmann F. et. al. **Pattern-oriented software architecture - a system of patterns**. Wiley, 1996.
- Coplien, J. **Advanced C programming styles and idioms**. Reading-MA: Addison-Wesley, 1992.
- Shvets, Alexander. **Dive into design patterns**, Refactoring.Guru, 2018, available for download at: <https://www.goodreads.com/book/show/43125355-dive-into-design-patterns>
- Braude, Eric. **Projeto de Software: Da programação à arquitetura: Uma abordagem baseada em Java**, Bookman, 2005.

Thank you!

Prof. Dr. Rosana T. Vaccare Braga | www.linkedin.com/in/rosana-braga-13778912

MBA **USP**
ESALQ